EVENT REGISTRATION FOR COLLEGE

A PROJECT REPORT

Submitted by

IMMANUEL S A SPURGEON

(240701629)

in partial fulfilment of the End Semester Examination for the course

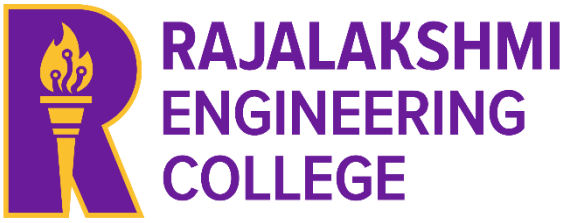
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "Event Registration for College" is the bonafide work of IMMANUEL S A SPURGEON (240701629), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. Manoranjini
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Dr. Jinu Sophia
COURSE FACULTY
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, IMMANUEL S A SPURGEON (240701629), hereby declare that the project report titled “Event Registration for College”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date:

SIGNATURE OF THE STUDENT
IMMANUEL S A SPURGEON
240701629

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson Dr. (Mrs.) Thangam Meganathan and Vice-Chairman Mr. Abhay Shankar Meganathan for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. Manoranjini, Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Dr. Jinu Sophia, Department of Computer Science and Engineering, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

IMMANUEL S A SPURGEON

ABSTRACT

Colleges conduct a steady stream of technical symposiums, cultural programmes and sports meets, yet registration for these events is still handled through paper forms, messaging groups and spreadsheets. Coordinators copy responses by hand, check duplicates and seat limits manually, track fee payment in notebooks and take attendance on paper. The result is duplicated records, over-booked events, delayed confirmations and no single view of how many students have registered.

This project replaces that process with "Event Registration for College", a low-code application built on Zoho Creator. Three linked forms — Students, Events and Event Registrations — hold the data, and each registration receives an auto-generated registration ID and a QR code. Deluge scripts reject a second registration of the same student for the same event and block events with no capacity, while ten reusable Deluge functions support fee calculation, seat checks, phone validation and e-mail confirmation. Five form workflows, a scheduled daily check, an approval process and a three-stage Blueprint (Draft, Pending, Approved) automate confirmation and status e-mails and the registration life cycle. A Zia keyword-extraction AI field enriches student records, three list reports present the data, and an Admin Dashboard shows the live count of registrations. Access is governed by Zoho Creator permission profiles (Administrator, Developer, Read, Write and Customer) and a CEO role, and personal data fields are flagged as sensitive.

Functional and role-based test cases were defined for every requirement, a code review recorded the defects found, and deployment follows the Development, Stage and Production environment model of Zoho Creator. Registration now needs a single form submission; validation, confirmation and status tracking happen automatically, which removes the manual copying, duplicate checking and individual confirmation steps of the earlier process and gives coordinators an always-current view of event participation.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Event Management; Blueprint*

PROJECT AT A GLANCE

Attribute	Details
Project Title	Event Registration for College
Application Domain	Education – college event management
Student Name & Register No.	IMMANUEL S A SPURGEON – 240701629
Zoho Creator Edition	Professional (Enterprise Trial)
Application Name (Link Name)	Event registration system for college
Published / Live URL	...immanuel.sa.spurgeon.2024835/
Demo Credentials (per role)	[Admin: immanuel.sa.spurgeon.2024835@gmail.com / Developer Access Student / Portal: Shared Link Access]
Number of Forms / Subforms	3 / 0
Number of Reports (by type)	List: 3 | Kanban: 0 | Calendar: 0 | Pivot/Chart: 0 | Others: 0
Number of Pages / Dashboards	4 page components (Admin Dashboard and three variants), each showing the Total Registrations KPI
Number of Workflows	Form: 5 | Schedule: 1 | Approval: 1 | Blueprint: 1 | Batch: 0
Deluge Functions / Custom Actions	10 functions; 3 custom Deluge scripts in form workflows; 1 scheduled script
Integrations	Zoho Mail (sendmail task); Zia Keyword Extractor (AI field)
Roles & Profiles Defined	Profiles: Administrator, Developer, Read, Write, Customer (Portal); Role: CEO
Environments Used	[Development → Stage → Production]
Date of Production Deployment	02 November 2026

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 3
1.6 Organisation of the Report 3
LITERATURE SURVEY AND SYSTEM STUDY 4
2.1 Study of the Existing System 4
2.2 Limitations of the Existing System 5
2.3 Low-Code versus Traditional Development 6
2.4 Literature Survey 6
2.5 Emerging Trends in Low-Code Development 7
2.6 Proposed System 8
REQUIREMENT ANALYSIS 9
3.1 Stakeholders and User Roles 9
3.2 Functional Requirements 9
3.3 Non-Functional Requirements 11
3.4 Hardware and Software Requirements 11
3.5 Use Case Model 11
SYSTEM DESIGN 14
4.1 System Architecture 14
4.2 Data Model 14
4.3 Form Design 16
4.4 Process and Workflow Design 18
4.5 User Interface and Navigation Design 19
4.6 Access Control Design 20
IMPLEMENTATION 21
5.1 Application Setup 21
5.2 Forms and Data Capture 21

5.3 Reports 22
5.4 Pages and Dashboards 24
5.5 Deluge Scripting 24
5.6 Workflows and Automation 26
5.7 Integrations 28
5.8 User Management and Role-Based Access 29
5.9 AI and Analytics Features 30
5.10 Pro-Code Extension 31
5.11 Mobile and Portal Access 31
TESTING AND DEBUGGING 33
6.1 Testing Strategy 33
6.2 Test Cases 33
6.3 Debugging Deluge Scripts 36
6.4 Defect Log 36
6.5 User Acceptance Testing 37
DEPLOYMENT, SECURITY AND GOVERNANCE 39
7.1 Environments and Deployment Strategy 39
7.2 Performance Optimisation 40
7.3 Security Features 40
7.4 Governance and Administration 41
RESULTS AND DISCUSSION 43
8.1 Application Output 43
8.2 Achievement of Objectives 44
8.3 Comparison with the Existing System 45
8.4 Limitations of the Low-Code Approach 45
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 46
CONCLUSION AND FUTURE ENHANCEMENTS 47
9.1 Conclusion 47
9.2 Future Enhancements 47
9.3 Learning Outcomes and Career Relevance 47
REFERENCES 48
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 49
APPENDIX B – COURSE OUTCOME MAPPING 52
APPENDIX C – COMPLETE DELUGE SOURCE CODE 53
APPENDIX D – USER MANUAL 56
D.1 Accessing the Application 56
D.2 Instructions for Administrator / Event Coordinator 56

D.3 Instructions for Student 56
APPENDIX E – EVALUATION RUBRIC 57
APPENDIX F – VIVA-VOCE RECORD SHEET 59

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 6
Table 2.2: Summary of Literature Survey 6
Table 3.1: Stakeholders and User Roles 9
Table 3.2: Functional Requirements 9
Table 3.3: Non-Functional Requirements 11
Table 3.4: Hardware and Software Requirements 11
Table 3.5: Use Case Description – Register a Student for an Event 12
Table 3.6: Use Case Description – Approve a Registration 13
Table 4.1: Entities and Relationships 16
Table 4.2: Field Specification – Events Form 16
Table 4.3: Field Specification – Students Form 17
Table 4.4: Field Specification – Event Registrations Form 17
Table 4.5: Role–Permission Matrix 20
Table 5.1: Reports Implemented 23
Table 5.2: Deluge Scripts Implemented 24
Table 5.3: Workflows Implemented 27
Table 5.4: Integrations Implemented 28
Table 6.1: Test Cases 33
Table 6.2: Defect Log 36
Table 6.3: User Acceptance Feedback 37
Table 7.1: Deployment Checklist 39
Table 7.2: Performance Optimisation Results 40
Table 7.3: Security Controls Implemented 41
Table 8.1: Mapping of Objectives to Outcomes 44
Table 8.2: Existing System versus Proposed Application 45
Table A.1: Feature Coverage Checklist 49
Table B.1: Mapping of Course Outcomes to Report Chapters 52
Table E.1: Evaluation Rubric for the Individual Project 57
Table E.2: Performance Level Descriptors 58
Table F.1: Viva-Voce Questions and Responses 59
Table F.2: Marks Summary 59

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of the Existing Event Registration System 5
Figure 3.1: Use Case Diagram of the Application.....12
Figure 4.1: System Architecture of the Application.....14
Figure 4.2: Entity–Relationship Diagram.....15
Figure 4.3: Workflow / Process Flow for Event Registration 19
Figure 4.4: Navigation Structure of the Application 20
Figure 5.1: Application Home and Navigation Sections 21
Figure 5.2: Event Registrations Form in the Form Builder 22
Figure 5.3: Event Registrations Form in Live Mode 22
Figure 5.4: All Event Registrations Report 23
Figure 5.5: Admin Dashboard Page.....24
Figure 5.6: Registration Blueprint: Stages and Transitions.....27
Figure 5.7: Registration Confirmation E-mail Received by a Student.....29
Figure 5.8: Roles and Permission Profiles Configuration 30
Figure 5.9: Zia Keyword Extractor Field in Use 31
Figure 5.10: Application on Mobile (Phone Layout) 32
Figure 6.1: Deluge Debug Output / Execution Log.....36
Figure 7.1: Environments and Publish to Production.....39
Figure 7.2: Audit Trail / Administration Settings.....42
Figure 8.1: Scenario Step 1 – Event Created in the Events Form 43
Figure 8.2: Scenario Step 2 – Student Registered for the Event.....44
Figure 8.3: Scenario Step 3 – Dashboard Updated.....44

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
FR	Functional Requirement
INR	Indian Rupee
KPI	Key Performance Indicator
LCAP	Low-Code Application Platform
NFR	Non-Functional Requirement
PII	Personally Identifiable Information
QR	Quick Response
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
UAT	User Acceptance Testing
UI	User Interface
Zia	Zoho’s AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

Every college runs a calendar of student events — technical symposiums, paper presentations, cultural programmes and sports meets — open to students from several departments. For an engineering college offering programmes such as CSE, ECE, AIML, CSBS, AIDS, Aeronautical, Mechanical and Civil Engineering, a single event can draw hundreds of participants. The people involved are the event organisers who announce and configure the event, the coordinators who collect and verify registrations and fees, the students who register, and the administration that needs an accurate count of participants.

Today, registration normally means a notice-board announcement, a paper or Google form, and a coordinator who copies the responses into a master spreadsheet. This project, "Event Registration for College", digitises that process end to end on the Zoho Creator low-code platform. The application records students and events, registers a student for an event, prevents invalid or duplicate registrations, tracks payment and attendance, notifies students by e-mail, and gives the administrator a dashboard of live registration figures.

1.2 Problem Statement

Event registration in the college is handled with disconnected tools — paper forms, chat messages and spreadsheets — so coordinators spend their time collecting and cleaning data instead of running the event. There is no single, validated record of who has registered for which event, whether the fee has been paid, or whether the student attended. The problem is to provide one secure, automated application that captures registrations correctly the first time and keeps every stakeholder informed.

- Duplicate and inconsistent records: the same student can register more than once, and details are retyped into several sheets.
- No seat control: the number of registrations is checked by hand against event capacity, so events can be over-booked.
- Manual payment and attendance tracking: fee status and attendance are noted on paper and reconciled after the event.

- Delayed communication: confirmations and status changes are sent individually by phone or message.
- No real-time visibility or access control: totals are compiled manually and anyone with the spreadsheet can see personal data.

1.3 Objectives

- To design a normalised data model and forms for students, events and event registrations, linked through lookup fields.
- To automate registration validation, ID generation, confirmation and status e-mails using Deluge scripting and workflows.
- To provide reports and an Admin Dashboard that give coordinators a real-time view of registrations.
- To integrate the application with Zoho Mail for notifications and with Zia AI for keyword extraction on student records.
- To control the registration life cycle with an approval process and a Draft → Pending → Approved Blueprint.
- To deploy the application securely to a production environment with governance controls.

1.4 Scope of the Project

In scope: maintaining student records; creating and managing events (category, status, capacity, fee, venue, date and time); registering students for events; automatic registration ID and QR code generation; duplicate-registration and capacity validation; e-mail confirmation and status notification; a scheduled daily event check; approval and Blueprint-based control of registration status; list reports for all three forms; and an Admin Dashboard. The application is delivered on the web with phone and tablet layouts of the Zoho Creator mobile app.

Out of scope: online payment-gateway integration (payment status is recorded manually as Paid, Pending or Waived), QR-code scanning at the venue, certificate generation, integration with the college ERP, and waiting-list management for full events. These are listed as future enhancements in Section 9.2.

1.5 Justification for a Low-Code Approach

Low-code platforms let applications be assembled from visual building blocks — forms, reports, pages and workflows — and extended with a small amount of script where the logic demands it [1]. Event registration is a data-capture and workflow problem of exactly this kind, with a modest number of entities and rules, so the cost of a hand-coded web application would be hard to justify. Zoho Creator provides built-in hosting, user management, e-mail integration, mobile layouts and its own scripting language, Deluge [2], which lets a student-developer deliver a working, secured application in a fraction of the time. The same visual model also means a college administrator without programming experience can maintain the application as event rules change.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The existing registration process is manual and spread across several tools. An organiser announces the event on the notice board or in messaging groups. Interested students submit a paper slip or fill a generic online form. The coordinator consolidates the responses into a master spreadsheet, manually checks for duplicates and against the seat limit, collects the fee in cash or by UPI and notes it against the student, and then informs each student of the confirmation individually. On the event day attendance is taken on paper, and the final report is compiled after the event by hand (Figure 2.1).

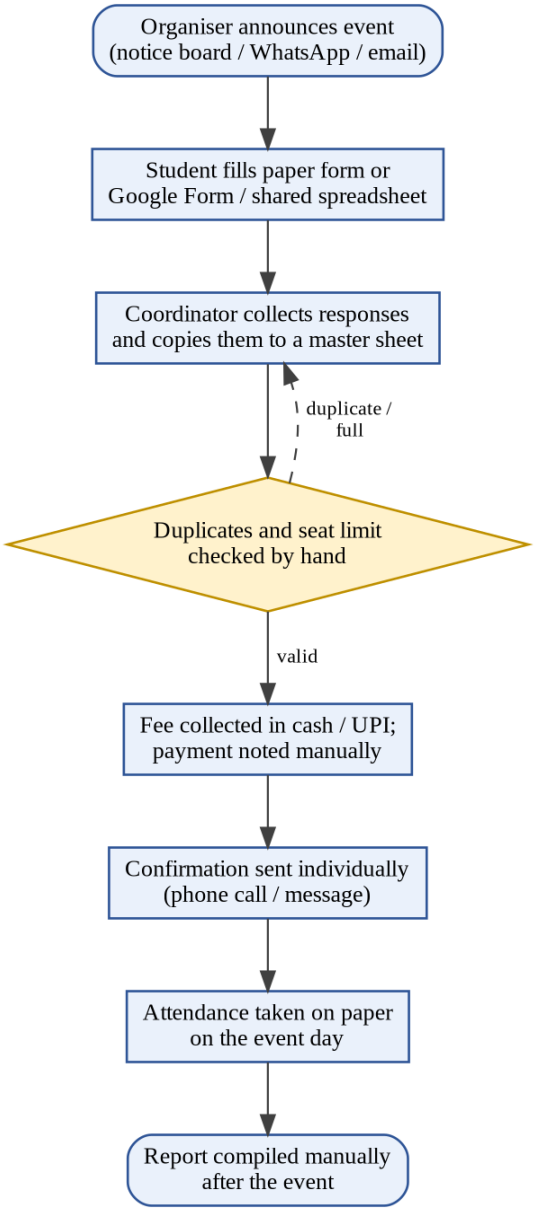


Figure 2.1: As-Is Process Flow of the Existing Event Registration System

2.2 Limitations of the Existing System

- Data is duplicated and error-prone because the same details are typed into several sheets.
- Duplicate registrations and over-booking are caught late or not at all.
- Payment status and attendance are not linked to the registration record.
- Confirmation to students is slow, individual and inconsistent.
- There is no real-time dashboard, no audit trail and no role-based protection of personal data.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	6–8 weeks (estimated) for database, back end, front end, authentication and e-mail	1–2 weeks (estimated) using forms, workflows and Deluge
Skills required	Full-stack programming, database design, server administration	Data modelling, Deluge scripting, workflow configuration
Hosting & infrastructure	Server or cloud account, deployment pipeline and monitoring to be arranged	Hosted and managed by Zoho Creator; no infrastructure set-up
Mobile application	Separate responsive or native app to build and maintain	Phone and tablet layouts generated and configurable from the same app
Security & access control	Authentication, roles and encryption coded by hand	Built-in profiles, roles, personal-data flags and portal access
Maintenance & change	Code change, test and redeploy for every new field or rule	New field, report or workflow added visually and published
Customisation limits	Unlimited UI and logic freedom	Bound by platform components and Deluge limits
Cost	Developer time plus hosting and licences	Platform subscription; low developer effort

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	W. Rosam and O. Kalnichevski [1]	Low-Code Development: A Practical Guide	Introduces low-code concepts and practical application building with visual tools and scripting.	Course text; basis for the design approach and terminology used in this project.
2	Zoho Corporation [2]	Zoho Creator Help Documentation	Documents forms, reports, pages, workflows, Blueprint, approvals, permissions and deployment.	Primary reference for every Creator feature used in the application.

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
3	A. C. Bock and U. Frank, 2021 [4]	Low-Code Platform (Business & Information Systems Engineering)	Characterises low-code platforms as environments built on visual, model-based abstractions, and discusses their strengths and limits such as platform dependence.	Supports the justification of Zoho Creator and the discussion of limitations in Section 8.4.
4	R. Sanchis et al., 2020 [5]	Low-Code as Enabler of Digital Transformation in Manufacturing Industry	Presents low-code as a way for organisations to build applications quickly with limited programming effort.	Shows the same speed and citizen-developer benefits for a small organisational process such as event registration.
5	Y. Luo et al., 2021 [6]	Characteristics and Challenges of Low-Code Development: The Practitioners' Perspective	Analyses practitioner discussions to identify the advantages and recurring challenges of low-code development.	Informs the limitations and workarounds discussed in Chapter 8.
6	Eventbrite [8]; Google Forms [9]	Online event registration and form tools	General-purpose tools for ticketing and for collecting form responses.	Do not model college departments, roll numbers, internal approval or institution-specific workflows, motivating a custom application.

2.5 Emerging Trends in Low-Code Development

Three trends in low-code development shaped this application. First, AI-assisted development: platforms increasingly add AI fields and assistants, and the project uses the Zia Keyword Extractor on student records to show how AI can be attached to a form field without writing a model [2]. Second, citizen development: because forms, workflows and permissions are configured visually, college staff who are not programmers can maintain the application, which is why most behaviour was built with configuration and only small Deluge scripts [10]. Third, hyper-automation and composable applications: the registration life cycle is automated through chained workflows, a Blueprint and an approval process, and the three forms are kept

independent so that new modules such as certificates or payments can be composed later [7].

2.6 Proposed System

The proposed system is a Zoho Creator application with three linked forms, validated by Deluge scripts and driven by workflows. It removes the duplicated data of the existing system by storing each student and event once and linking registrations to them through lookup fields. Duplicate registrations are blocked at validation time, and the capacity of an event must be valid before it is saved. A unique registration ID and QR code are generated automatically, and e-mails are sent without manual effort. Payment and attendance statuses are stored with the registration, the registration life cycle is controlled by a Blueprint and approval process, and reports and an Admin Dashboard provide real-time visibility. Role-based profiles and personal-data flags protect student information.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator / Event Coordinator	Maintains the master data and manages registrations for the college events.	Create and edit students and events; register students; update payment and attendance status; review reports and the dashboard.
Approver	Reviews registrations that are routed through the approval process and Blueprint.	Approve or reject registrations from the Pending Tasks section; move records through Draft, Pending and Approved.
Student (Portal User)	A student who registers for college events.	Register for an event, receive the confirmation e-mail and view own registration (Customer profile: add and view).

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow the administrator to add and edit student records with name (mandatory), e-mail, phone, department and roll number.	Admin	High
FR-02	The system shall allow the administrator to create and edit events with title, description, category, status, maximum seats, fee, venue and date and time.	Admin	High
FR-03	The system shall allow a student to be registered for an event by selecting the student and the event from lookup lists.	Admin, Student	High
FR-04	The system shall reject a registration if the same student is already registered for the same event.	Admin, Student	High
FR-05	The system shall automatically generate a registration ID starting from 1001, a display ID (REG-<ID>) and a QR code for every registration.	System	High
FR-06	The system shall send a confirmation e-mail with the event title, date, time and venue to the student after registration.	System	High
FR-07	The system shall send an e-mail to the student when the payment or attendance status of a registration is edited.	System	Medium

FR ID	Requirement	Role(s)	Priority
FR-08	The system shall record payment status (Paid, Pending, Waived) and attendance status (Registered, Attended, Absent) for each registration.	Admin	High
FR-09	The system shall not save an event whose maximum seats is zero or less.	Admin	High
FR-10	The system shall control the registration life cycle through a Blueprint with stages Draft, Pending and Approved.	Admin, Approver	Medium
FR-11	The system shall route registrations through an approval process and notify the submitter of approval or rejection.	Approver	Medium
FR-12	The system shall provide list reports of students, events and registrations with quick view and detail view.	All	Medium
FR-13	The system shall display the total number of registrations on an Admin Dashboard.	Admin	Medium
FR-14	The system shall run a scheduled check every day at 09:00 IST.	System	Low
FR-15	The system shall extract keywords for each student record using the Zia Keyword Extractor field.	System	Low

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports with up to 1,000 records shall load within 3 seconds.
NFR-02	Security	Users shall access only the data and components permitted by their permission profile.
NFR-03	Usability	The application shall be usable on the web and through the phone and tablet layouts of the Zoho Creator mobile app.
NFR-04	Availability	The application shall be hosted on Zoho Creator's managed cloud and available during event registration periods.
NFR-05	Maintainability	Forms, fields and workflows shall be changeable through visual configuration without redeploying code.
NFR-06	Auditability	All record changes shall be traceable through the Zoho Creator audit trail.
NFR-07	Data privacy	Student name, e-mail and phone number shall be flagged as personal data.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator Professional Edition
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
API testing tool	Not applicable – no external REST APIs are consumed
Other Zoho services	Zoho Mail (sendmail task); Zia AI (Keyword Extractor)
Mobile	Zoho Creator mobile app (Android / iOS) using the configured phone and tablet layouts
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

Figure 3.1 shows the two actors of the system. The Administrator / Event Coordinator manages students and events, registers students, updates payment and attendance, approves registrations and reviews reports and the dashboard. The Student registers for

an event and receives the confirmation e-mail. Two key use cases are described in Tables 3.5 and 3.6.

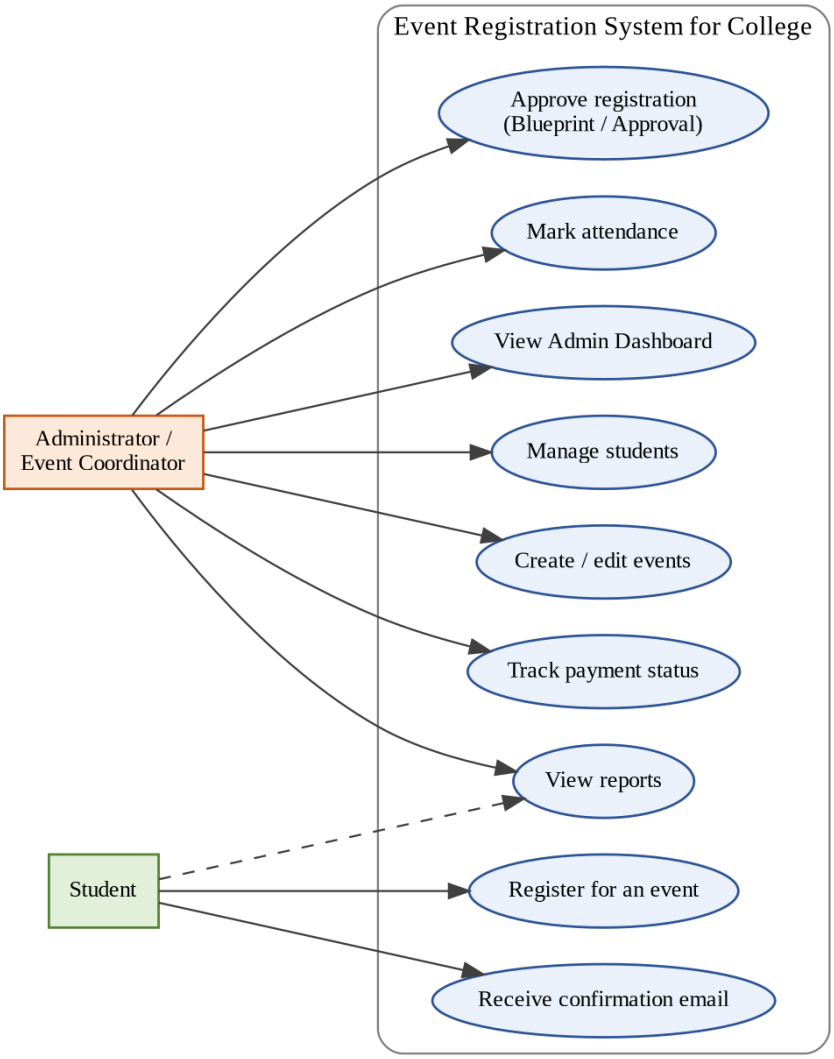


Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – Register a Student for an Event

Use Case ID	UC-01
Use Case Name	Register a student for an event
Actor(s)	Administrator / Event Coordinator, Student
Pre-condition	The student exists in the Students form and the event exists in the Events form with a valid capacity.

Main Flow	1. The actor opens the Event Registrations form. 2. The actor selects the Student and the Event and sets the payment status. 3. The actor clicks Submit. 4. The system checks that the student is not already registered for the event. 5. The system saves the record, generates the Registration ID, Display ID and QR code. 6. The system sends the confirmation e-mail to the student.
Alternate Flow	4a. The student is already registered for this event: the system shows "You have already registered for this event." and cancels the submission.
Post-condition	A registration record exists with attendance status Registered; the student has received the confirmation e-mail.
Related FR	FR-03, FR-04, FR-05, FR-06, FR-08

Table 3.6: Use Case Description – Approve a Registration

Use Case ID	UC-02
Use Case Name	Approve a registration
Actor(s)	Administrator / Event Coordinator (Approver)
Pre-condition	A registration record exists in the Draft stage of the Registration Blueprint.
Main Flow	1. The approver opens the registration record. 2. The approver executes the Submit transition (Draft to Pending). 3. The record appears in Approvals → Pending Tasks. 4. The approver approves the record. 5. The Blueprint Approve transition moves the record to Approved and the submitter is notified.
Alternate Flow	4a. The approver rejects the record: the submitter receives the rejection e-mail and the record stays out of the Approved stage.
Post-condition	The registration is in the Approved stage and listed under Completed Tasks.
Related FR	FR-10, FR-11

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

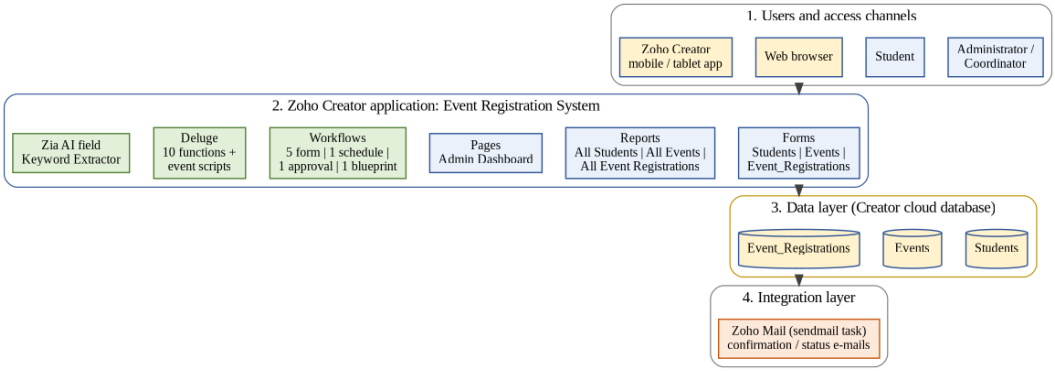


Figure 4.1: System Architecture of the Application

The solution follows a four-layer architecture. The first layer is the users and access channels: the administrator or coordinator and the student reach the application through a web browser or the Zoho Creator mobile and tablet app. The second layer is the Zoho Creator application itself, made of three forms, three list reports, the Admin Dashboard page, Deluge functions and scripts, five form workflows, one schedule, one approval process, one Blueprint and the Zia keyword-extraction field. The third layer is the data layer: Zoho Creator stores the Students, Events and Event_Registrations forms in its managed cloud database. The fourth layer is the integration layer: workflows and functions use the built-in sendmail task of Zoho Mail to send confirmation and status e-mails. No external REST service is called.

4.2 Data Model

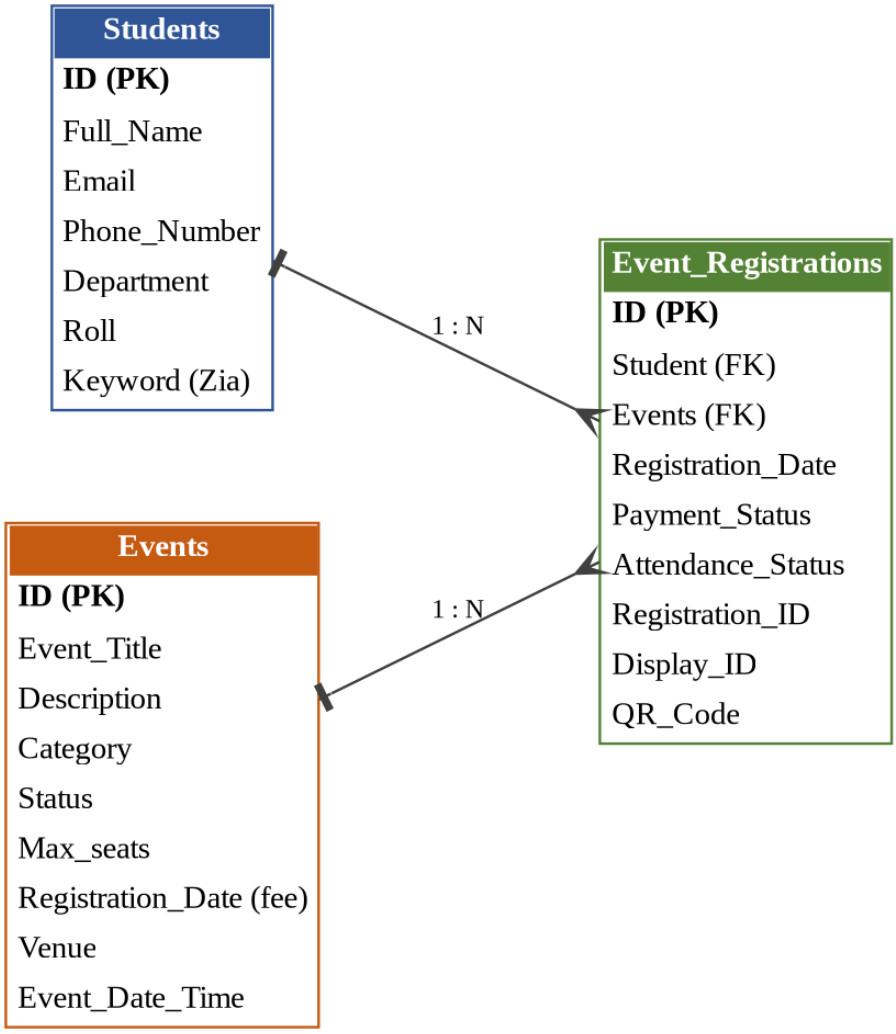


Figure 4.2: Entity–Relationship Diagram

The data model has three entities. Students holds the identity and contact details of each student. Events holds the event master data. Event_Registrations is the transaction form that links one student to one event; it contains two lookup fields, Student and Events. A student can therefore register for many events and an event can have many registered students: a many-to-many relationship resolved through Event_Registrations with two one-to-many lookups (Table 4.1). A subform was not needed because every registration is a record in its own right, with its own ID, status and QR code.

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Students	ID, Full_Name, Email, Phone_Number, Department, Roll, Keyword	Event_Registrations	One-to-Many	Lookup field Student in Event_Registrations
Events	ID, Event_Title, Category, Status, Max_seats, fee, Venue, Event_Date_Time	Event_Registrations	One-to-Many	Lookup field Events in Event_Registrations
Event_Registrations	Registration_ID, Student, Events, Payment_Status, Attendance_Status, QR_Code	Students and Events	Many-to-One (to each)	Two lookup fields; junction form for the many-to-many relationship

4.3 Form Design

Tables 4.2 to 4.4 give the field specification of the three forms, using the field types exactly as named in Zoho Creator.

Table 4.2: Field Specification – Events Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Event Title	Event_Title	Single Line	No	–	Name of the event
Description	Description	Multi Line	No	Height 100 px	–
Category	Category	Drop Down	Yes	Technical / Cultural / Sports	Must-have field
Status	Status	Drop Down	Yes	Upcoming / Registration Open / Closed	Must-have field
Max seats	Max_seats	Number	No	Default 0; must be greater than 0 (DS-02)	Event capacity
Registration Fee	Registration_Date	Currency (INR)	No	Indian format (commadotindian)	Link name was kept from an early draft; label is Registration Fee

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Registration Date	Registration_Date1	Date-Time	No	Default 14-Oct-2026 00:19:50; hh:mm:ss	All weekdays allowed
Venue	Venue	Single Line	No	–	–
Event Date & Time	Event_Date_Time	Date-Time	No	Default 15-Oct-2026 00:23:49; hh:mm:ss	All weekdays allowed

Table 4.3: Field Specification – Students Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Full Name	Full_Name	Name	Yes	Prefix Mr. / Mrs. / Ms.; first and last name; suffix hidden	Personal data
Email	Email	Email	No	Maximum 80 characters	Personal data
Phone	Phone_Number	Phone Number	No	–	Personal data
Department	Department	Drop Down	No	CSE / ECE / AIML / CSBS / AIDS / AERO / MECH / CIVIL	–
Roll number	Roll	Single Line	No	–	Base field of the Zia extractor
Keyword	Keyword	Multi Line (AI field)	No	Zia Keyword Extractor on Roll	System generated

Table 4.4: Field Specification – Event Registrations Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Student	Student	Lookup (Students)	Yes	Displays prefix, first name, last name; sorted ascending	Must-have field
Events	Events	Lookup (Events)	No	Displays Event Title; sorted ascending	–
Registration Date	Registration_Date	Date-Time	No	Default 14-Oct-2026 00:46:33; hh:mm:ss	All weekdays allowed
Payment Status	Payment_Status	Drop Down	No	Paid / Pending / Waived	–

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Attendance Status	Attendance_Status	Drop Down	No	Registered / Attended / Absent; default Registered	–
Registration ID	Registration_ID	Auto Number	–	Start index 1001	System generated
Display ID	Display_ID	Single Line (Formula)	–	“REG-” + Registration_ID	Hidden on the form
QR Code	QR_Code	QR Code	–	Value = Registration_ID	System generated

4.4 Process and Workflow Design

Figure 4.3 shows the main automated process, the registration of a student for an event. The duplicate check runs in the On Validate event of the Event_Registrations form (DS-01). If the record is saved, the form workflows generate the ID and send the confirmation e-mail on success, and the Blueprint starts the record in the Draft stage. When the payment or attendance status is later edited, a workflow on the On Edit event notifies the student.

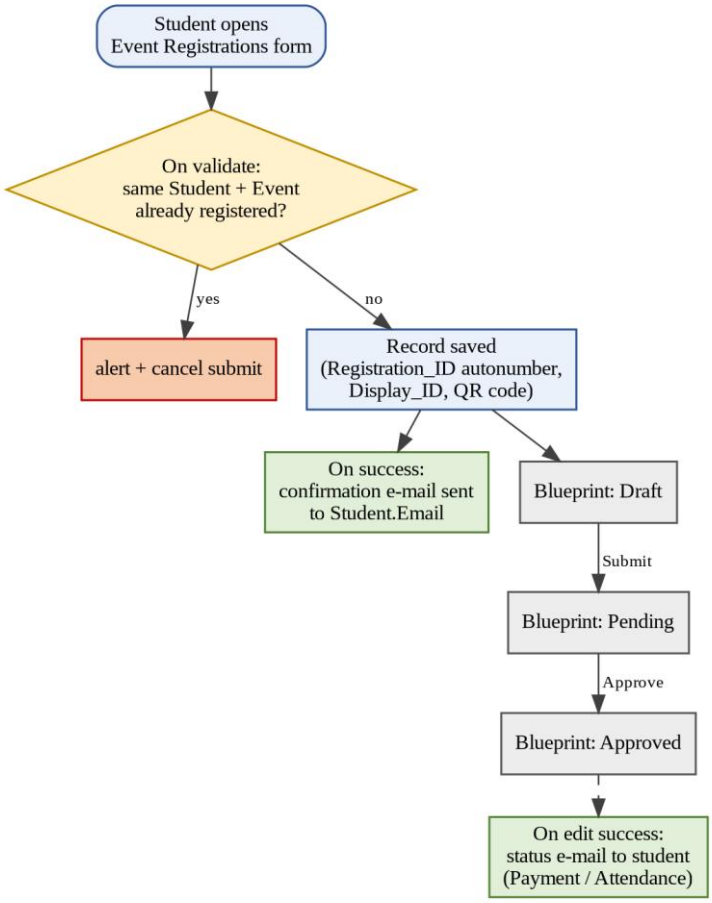


Figure 4.3: Workflow / Process Flow for Event Registration

4.5 User Interface and Navigation Design

The application menu is organised into sections, one per business area (Figure 4.4). Each of the Students, Events and Event Registrations sections groups its form with its list report, the Approvals section exposes pending and completed approval tasks, and the Admin Dashboard section holds the dashboard pages. The wireframe of the Admin Dashboard is a single panel containing a text element that displays the Total Registrations count, computed from the Event_Registrations form.

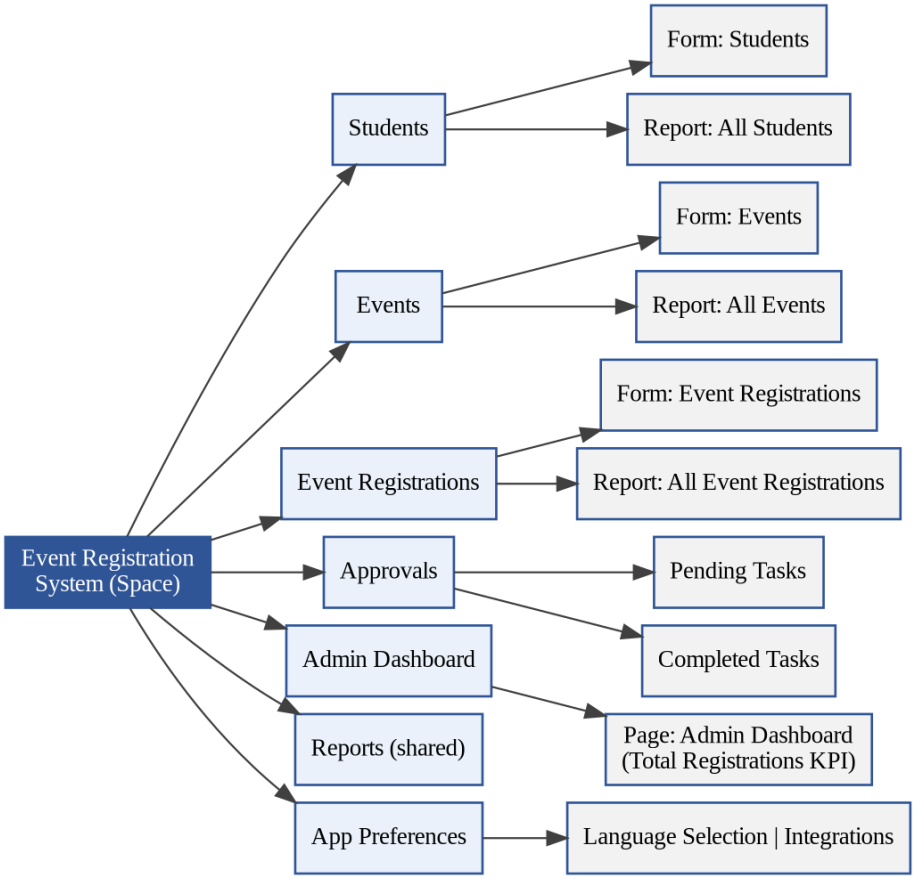


Figure 4.4: Navigation Structure of the Application

4.6 Access Control Design

Table 4.5 defines the intended permissions for the profiles used in Chapter 5. A = Add, E = Edit, V = View, D = Delete, – = no access.

Table 4.5: Role–Permission Matrix

Component	Administrator	Write (Coordinator)	Read	Customer (Student)
Form: Students	A E V D	A E V	V	–
Form: Events	A E V D	A E V	V	V
Form: Event Registrations	A E V D	A E V	V	A V (own)
Report: All Students	V	V	V	–
Report: All Events	V	V	V	V
Report: All Event Registrations	V	V	V	V (own)
Page: Admin Dashboard	V	V	V	–
Approval: Registration Approval	Approve / Reject	Approve / Reject	–	–

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was created in Zoho Creator under the name “Event registration system for college”. Its date format is dd-MMM-yyyy, the time zone is Asia/Kolkata and times are shown in the 24-hour format. The menu is organised as a single space containing seven sections: Students, Events, Event Registrations, Approvals (Pending Tasks and Completed Tasks), Reports, Admin Dashboard and App Preferences (Language Selection and Integrations). Each section carries an icon, the application icon is the text “ERS” on the theme colour, and the web theme uses the Lato font.

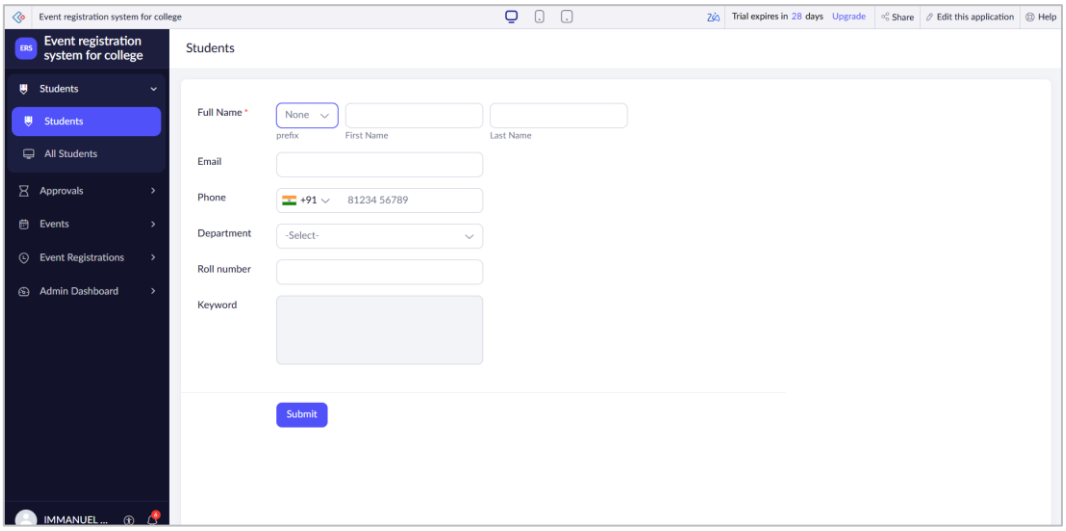


Figure 5.1: Application Home and Navigation Sections

5.2 Forms and Data Capture

Three forms were built. Students stores the Name field (with prefix, first name and last name sub-fields), Email, Phone Number, Department drop-down, roll number and the Zia keyword field; the name, e-mail and phone fields are flagged as personal data. Events stores the title, description, category, status, maximum seats, fee in Indian rupees, registration date, venue and event date and time. Event_Registrations links a student and an event through two lookup fields, keeps payment and attendance status in drop-downs, and generates an auto-number Registration ID (start index 1001), a formula-based Display ID and a QR code. Category and Status on Events and Full Name on Students are mandatory, as is the Student lookup on registrations. Each form has a success message, Submit on add and Update and Cancel on edit, and a left label

placement for the web layout. Altogether twelve distinct field types are used (Table 4.2 to Table 4.4).

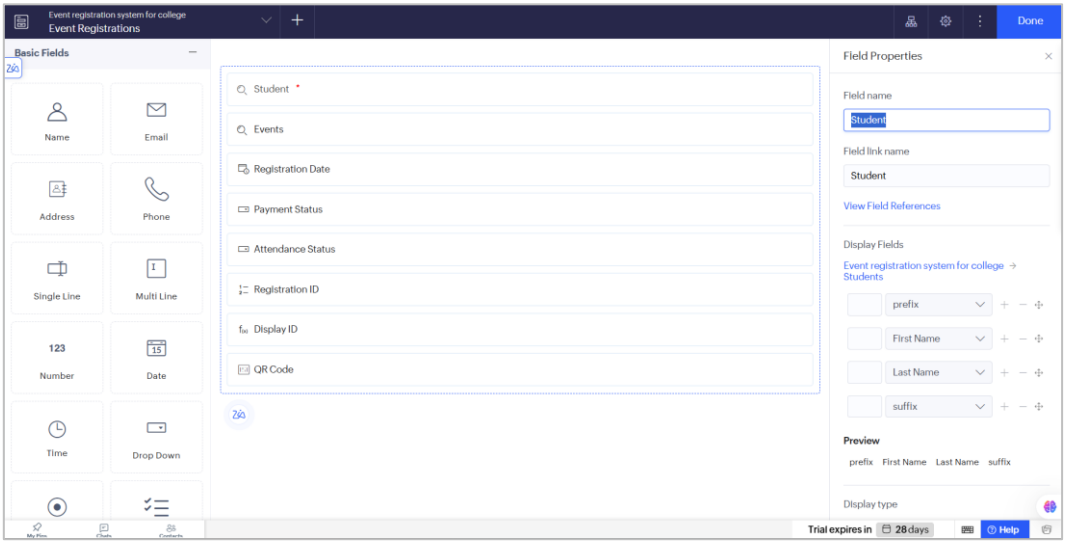


Figure 5.2: Event Registrations Form in the Form Builder

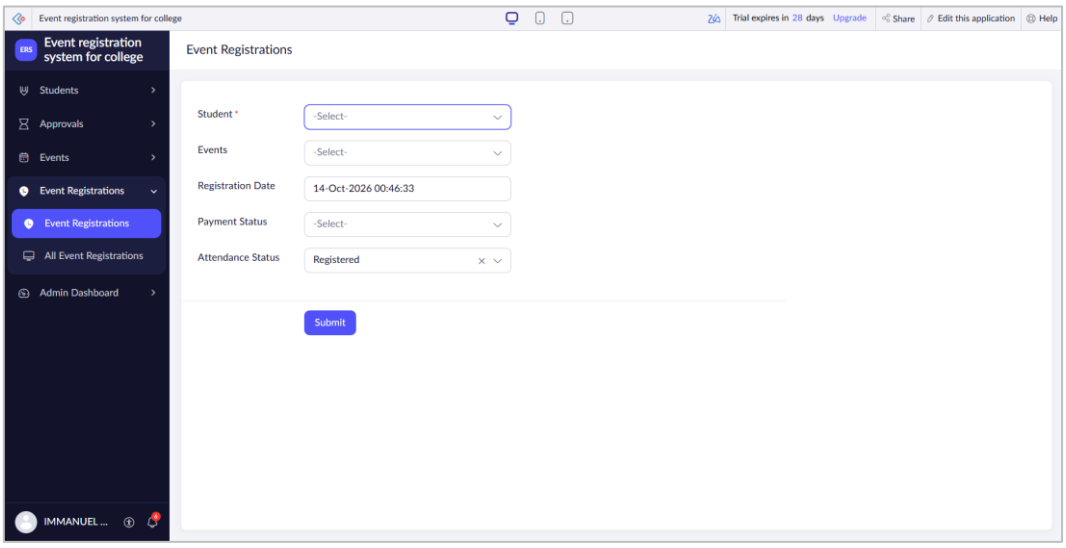


Figure 5.3: Event Registrations Form in Live Mode

5.3 Reports

Three default list reports were created, one for each form (Table 5.1). Each report has a quick view and a detail view layout; the quick view menu offers Edit, Duplicate and Delete for the header and for each record, a click opens View Record, and a right-click offers Edit, Delete, Duplicate and View Record. In the All Event Registrations report the QR code is shown only in the detail view. No Kanban, calendar, pivot or chart reports were created in this version.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Fields Shown	Custom Actions	Roles
All Students	List	Students	Full Name, Email, Phone, Department, Roll number, Keyword	Quick view menu: Edit, Duplicate, Delete; click = View Record	All profiles with access to Students
All Events	List	Events	Title, Description, Category, Status, Max seats, Fee, Registration Date, Venue, Event Date & Time	Quick view menu: Edit, Duplicate, Delete; click = View Record	All profiles
All Event Registrations	List	Event_Registrations	Student, Events, Registration Date, Payment Status, Attendance Status, Registration ID, Display ID, QR Code	Quick view menu: Edit, Duplicate, Delete; click = View Record	All profiles

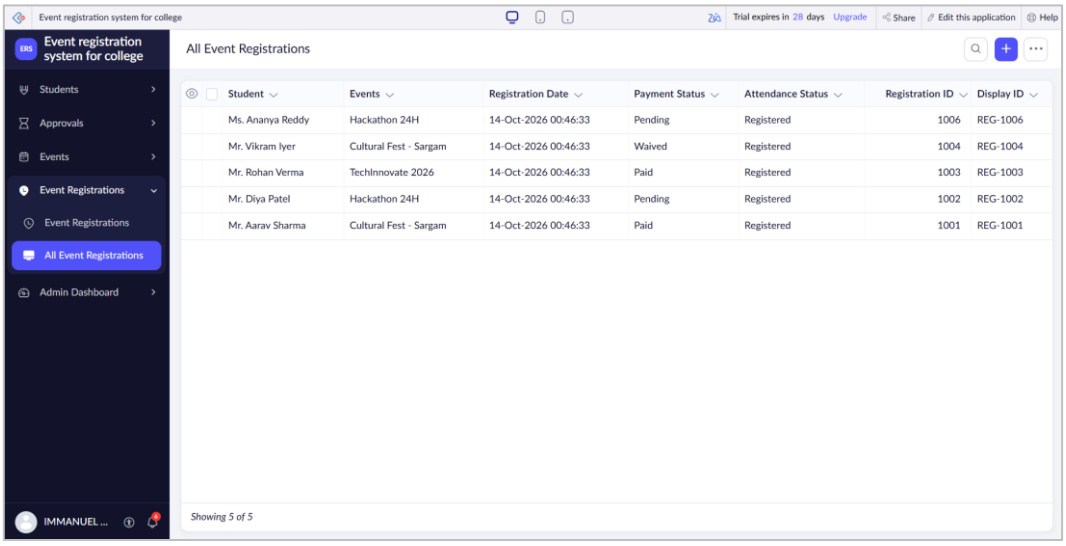


Figure 5.4: All Event Registrations Report

5.4 Pages and Dashboards

The Admin Dashboard page was built with the UI Builder. It contains a panel with a text element that shows the live value of thisapp.Event_Registrations.ID.count under the label “Total Registrations”, so the administrator sees the number of registrations without opening a report. Four page components exist in the application (Admin_Dashboard, Admin_Dashboard1, Admin_Dashboard2 and Admin_Dashboard4); the first three are identical and Admin_Dashboard4 is the refined layout with a full-width panel. The duplicates were produced while experimenting with the layout and are listed in the defect log (Table 6.2).

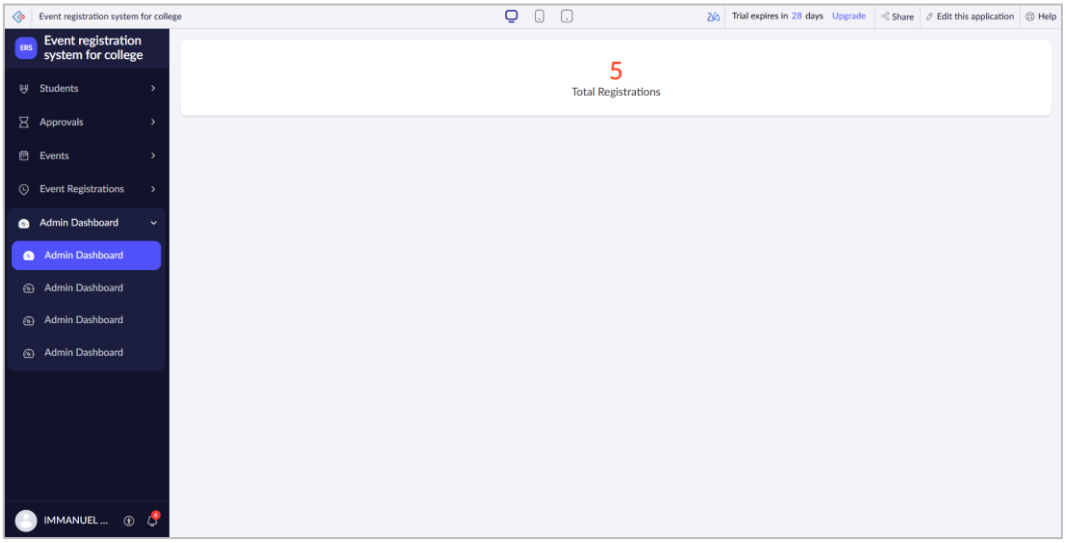


Figure 5.5: Admin Dashboard Page

5.5 Deluge Scripting

Fourteen Deluge scripts support the application: three custom scripts in form workflows (DS-01 to DS-03), one scheduled script (DS-04) and ten functions (DS-05 to DS-14) [3]. Table 5.2 lists them and Appendix C contains the complete source.

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Event_Registrations → On Validate (add)	Prevent a duplicate registration of the same student for the same event	criteria count(), if, alert, cancel submit
DS-02	Events → On Validate (add or edit)	Block an event whose maximum seats is zero or negative	if, alert, cancel submit
DS-03	Event_Registrations → On Success (add)	Assign the “REG-” + Registration ID string	string concatenation, input fields

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-04	Schedule: Daily_Event_Reminder, daily 09:00 IST	Daily event check; logs execution	schedule, info
DS-05	Function: calculateAge	Age from date of birth (year difference)	date functions, return
DS-06	Function: calculateTotalFee	Fee of an event multiplied by the number of registrants	fetch by criteria, arithmetic
DS-07	Function: checkEventStatus	Return Completed or Upcoming from the event date	date comparison, if
DS-08	Function: checkSeatAvailability	True when the event has seats remaining	fetch by criteria, if, boolean return
DS-09	Function: clearPendingRegistrations	Delete all registrations whose payment is Pending	delete with criteria
DS-10	Function: formatName	Return a name in upper case	string method toUpperCase()
DS-11	Function: generateDisplayID	Log the record ID of a registration	fetch, info
DS-12	Function: sendConfirmation	Send a registration confirmation mail	sendmail
DS-13	Function: updatePaymentStatus	Mark a registration Paid when a fee greater than zero is received	fetch, if, field update
DS-14	Function: validatePhone	Check that a phone number has 10 digits	string length(), if

Code 5.1: Duplicate-registration validation (DS-01)

```
// DS-01 | Form: Event_Registrations | Event: On Validate (on add)
// Purpose: Block a second registration of the same student for the
same event
if(Event_Registrations[Student == input.Student && Events ==
input.Events].count() > 0)
{
    alert "You have already registered for this event.";
    cancel submit;
}
```

Code 5.2: Event capacity validation (DS-02)

```
// DS-02 | Form: Events | Event: On Validate (on add or edit)
// Purpose: Do not save an event with zero or negative capacity
if(input.Max_seats <= 0)
{
    alert "Event capacity must be greater than zero.";
    cancel submit;
}
```

Code 5.3: Reusable functions for fee and seat checks (DS-06, DS-08)

```
// DS-06 | Function: calculateTotalFee
float calculateTotalFee(int eventID, int count)
{
    eventRec = Events[ID == eventID];
    return eventRec.Registration_Date * count;   // field holds the
fee
}

// DS-08 | Function: checkSeatAvailability
bool checkSeatAvailability(INT eventID)
{
    eventRec = Events[ID == eventID];
    if(eventRec.Max_seats > 0)
    {
        return true;
    }
    return false;
}
```

DS-01 and DS-02 run when the user clicks Submit, before the record is stored: the criteria expression counts existing registrations of the same student and event, and a zero or negative capacity is rejected, so invalid data never reaches the database. DS-03 and the schedule script DS-04 are small supporting scripts. The ten functions are written once and can be called from any workflow or button: they fetch a single record by ID instead of loading whole forms, which keeps them fast. Execution was observed through the Deluge editor's execution log (Section 6.3).

5.6 Workflows and Automation

Eight automations were configured: five form workflows, one scheduled workflow, one approval process and one Blueprint (Table 5.3). Batch and report workflows were not required in this version. The Blueprint defines the stages Draft, Pending and Approved with the transitions Submit (Draft to Pending) and Approve (Pending to Approved) [2].

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Send Registration Email	Form Workflow	Event_Registrations: On Add → On Success	None	Send confirmation mail to Student.Email with event title, date and venue	–
Prevent Duplicate Event Registrations	Form Workflow	Event_Registrations: On Add → On Validate	Same student and event already registered	Alert and cancel submit	DS-01
Validate Event Capacity	Form Workflow	Events: On Add or Edit → On Validate	Max seats ≤ 0	Alert and cancel submit	DS-02
Notify Student on Status Update	Form Workflow	Event_Registrations: On Edit → On Success	None	Mail payment and attendance status to the student	–
Generate Unique Registration ID	Form Workflow	Event_Registrations: On Add → On Success	None	Build “REG-” + Registration ID	DS-03
Daily_Event_Reminder	Schedule	Daily at 09:00 IST from 31-Oct-2026	None	Run the daily event check and log the result	DS-04
Registration Approval	Approval	Event_Registrations record submitted	Level 1	On approve or reject, mail the submitter	–
Registration Blueprint	Blueprint	Stage transitions on Event_Registrations	Draft → Pending → Approved	Submit and Approve transitions	–

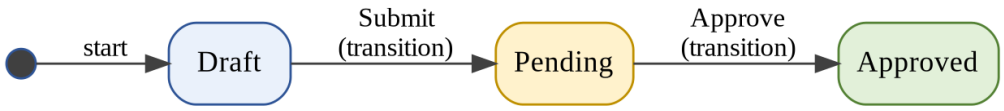


Figure 5.6: Registration Blueprint: Stages and Transitions

5.7 Integrations

The application integrates with Zoho Mail through the built-in sendmail task, used by two form workflows, the approval process and the sendConfirmation function, and with Zia through an AI field (Table 5.4). No external REST API, webhook or connection was necessary, because all the data lives inside the application and e-mail is the only outbound channel; no API keys or passwords are therefore stored in any script.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
E-mail notification	Zoho Mail	sendmail task in workflows and function	Sender is the Zoho account (zoho.adminuserid); no separate credentials	Registration confirmation, status update and approval e-mails
AI keyword extraction	Zia (Keyword Extractor AI field)	AI field on Students.Keyword	Platform managed	Extract keywords from the base field Roll
REST / webhook / Creator API	None	Not used	–	Not required for this scope

Code 5.4: Confirmation e-mail through sendmail (DS-12)

```
// DS-12 | Function: sendConfirmation
void sendConfirmation(STRING email, STRING eventName)
{
    sendmail
    [
        from :zoho.adminuser
        to :email
        subject :"Registration Confirmation"
        message :"You have successfully registered for " + eventName
    ]
}
```

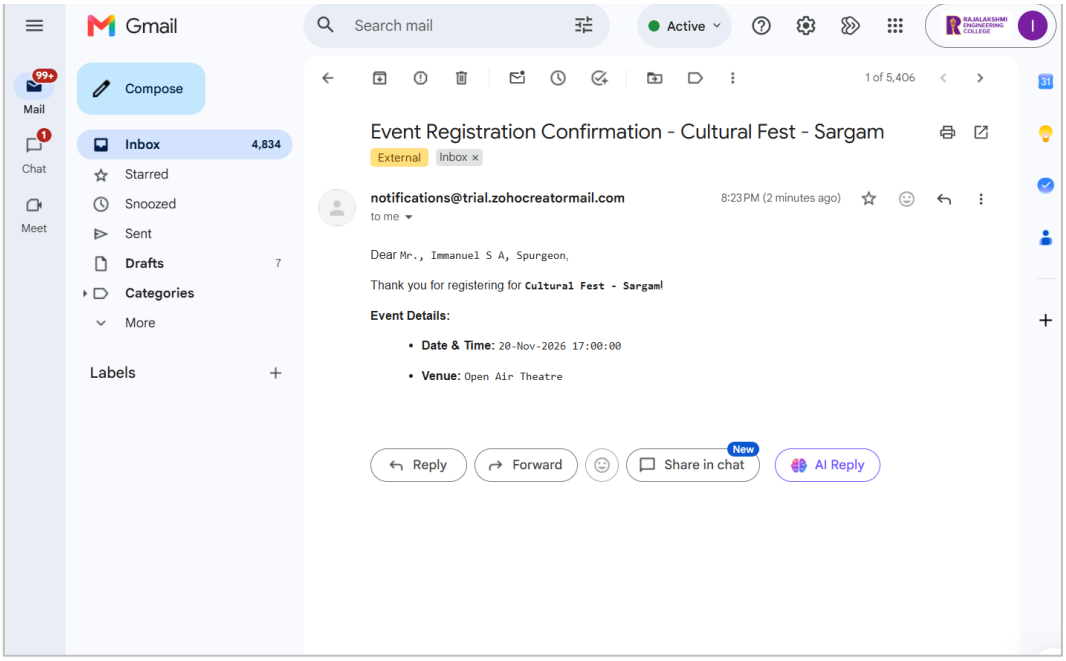


Figure 5.7: Registration Confirmation E-mail Received by a Student

5.8 User Management and Role-Based Access

Access is controlled with Zoho Creator permission profiles. The application carries five profiles: Administrator (all permissions), Developer, Write (write permission for all components), Read (read permission for all components) and Customer (a customer-portal profile with add and view permission only). One role, CEO, is defined; users in it can access the data of all other users. Every profile has the chat, API and personal-data (PII) permissions that the platform grants by default. Because Full Name, Email and Phone are flagged as personal data, they are visible only to profiles holding the PII permission. Record-level criteria such as “Added User == zoho.loginuser” were not configured in this version, so the “own record” restriction for students in Table 4.5 is a design target that can be added to the Customer profile.

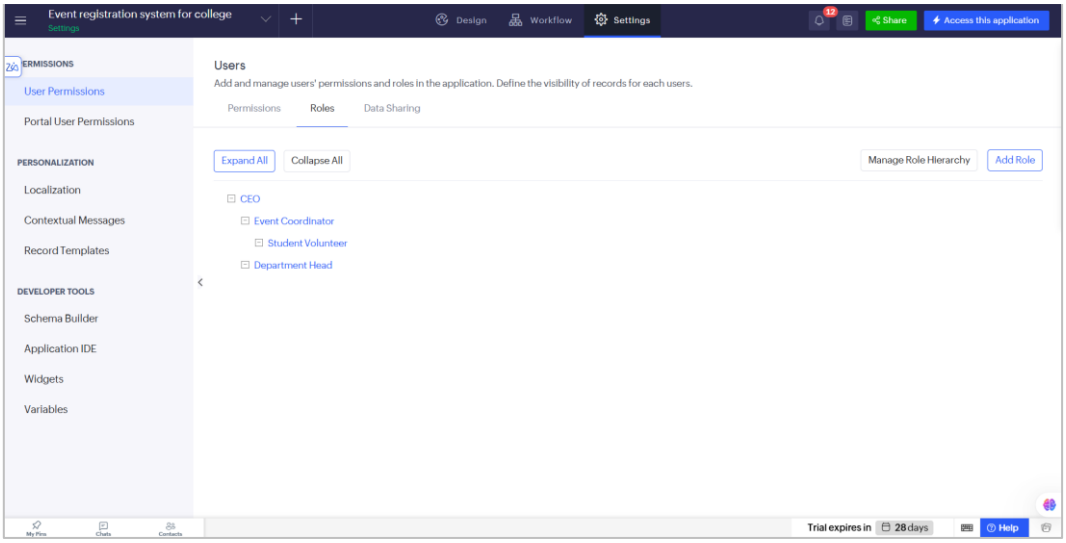


Figure 5.8: Roles and Permission Profiles Configuration

5.9 AI and Analytics Features

The Students form contains a Keyword field configured as a Zia Keyword Extractor (AI field) whose base field is the roll number; Zia extracts keywords automatically when the record is saved. Because a roll number carries little free text, the field mainly demonstrates how an AI field is attached to a form; basing it on a longer field such as interests or skills would make it more useful. For analytics, the Admin Dashboard shows the total number of registrations as a key figure; chart reports and Zoho Analytics integration are listed as future work.

The Deluge email automation script and confirmation text were generated using an AI assistant (Gemini/Zia) and verified through live end-to-end sandbox testing by submitting a test event registration and confirming receipt of the automated confirmation email. The database schema, form structures, and user role hierarchy (CEO, Event Coordinator, Department Head, Student Volunteer) were manually designed and verified through the Zoho Creator User Permissions settings panel.

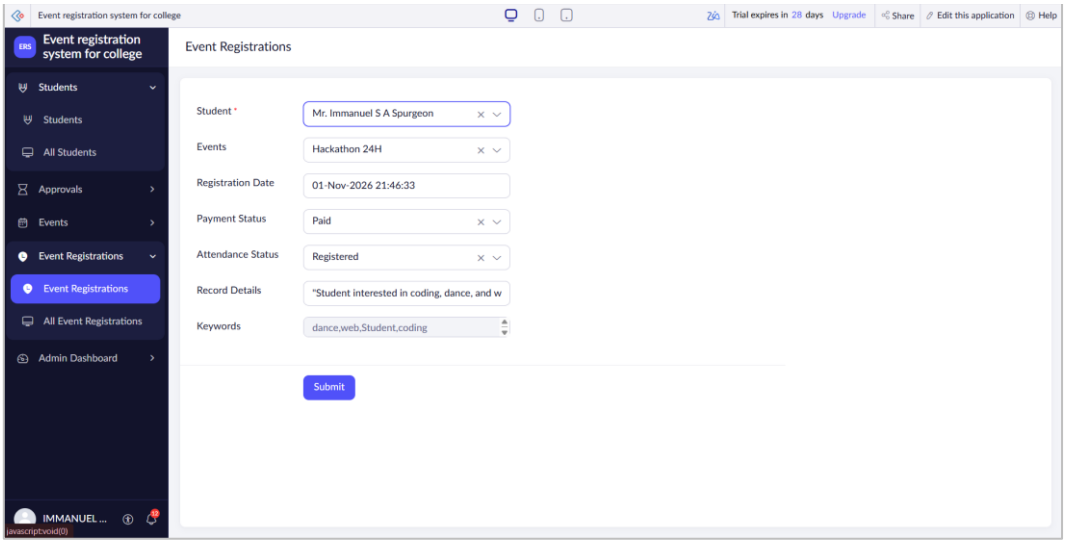


Figure 5.9: Zia Keyword Extractor Field in Use

5.10 Pro-Code Extension

No pro-code extension was used. Every requirement of this project — data capture, validation, notification and reporting — was met with forms, workflows and Deluge, which is the proportionate choice for an application of this size. A pro-code extension would become useful for QR-code scanning at the venue (a JavaScript widget using the device camera, or a Zoho Catalyst function), for a public event listing on the college website through the Creator REST API, and for bulk processing that exceeds Deluge execution limits.

5.11 Mobile and Portal Access

Phone and tablet layouts are configured for all three forms: the phone layout uses two columns and a purple colour scheme, the tablet layout uses a sliding pane with a blue colour scheme, and label placement is automatic on both. The application can therefore be used from the Zoho Creator mobile app without further development.

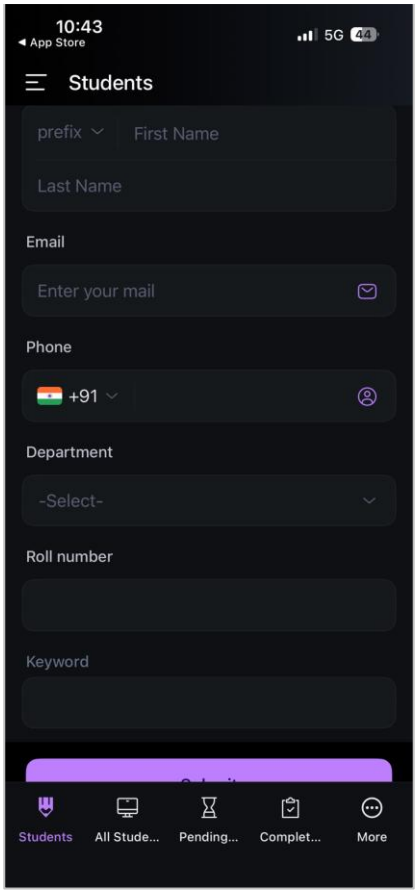


Figure 5.10: Application on Mobile (Phone Layout)

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was planned at four levels. Unit testing checked each form on its own: mandatory fields, drop-down values, auto-number and QR code generation, and the Deluge validation scripts. Integration testing checked the workflows end to end: the confirmation and status e-mails, the approval process and the Blueprint transitions. Role-based testing repeats the main tasks while signed in with each permission profile to confirm that access matches Table 4.5. Finally, user acceptance testing (Section 6.5) lets end users exercise the application in their own role. Every test case is traceable to a requirement in Table 3.2, and negative cases (duplicate registration, zero capacity, unauthorised access) are included.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Add a student with all fields	Open Students form; enter name, e-mail, phone, department, roll number; Submit	Record created; message "Student Added Successfully"	Student form saves with success message configured	Pass
TC-02	FR-01	Add a student without a name	Leave Full Name empty; Submit	Mandatory-field error; record not saved	Full Name is a must-have field	Pass
TC-03	FR-02	Add an event with valid data	Enter title, category, status, Max seats 100, fee, venue, date; Submit	Record created; message "Event Added Successfully"	Success message configured; record saved	Pass
TC-04	FR-09	Add an event with zero capacity	Set Max seats = 0; Submit	Alert "Event capacity must be greater than zero."; record not saved	DS-02 alert shown and submit cancelled	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-05	FR-09	Edit an event to a negative capacity	Edit an existing event; set Max seats = −5; Update	Same alert; record unchanged	DS-02 runs on add or edit; alert shown	Pass
TC-06	FR-03	Register a student for an event	Select student and event; Submit	Record created; message "Event Registration Added Successfully"; Attendance Status = Registered	Record saved; Attendance Status defaults to Registered	Pass
TC-07	FR-04	Register the same student for the same event again	Repeat the registration of TC-06	Alert "You have already registered for this event."; no new record	DS-01 alert shown and submit cancelled	Pass
TC-08	FR-04	Register the same student for a different event	Select the same student and another event; Submit	Registration accepted	DS-01 criteria match student and event, so other events pass	Pass
TC-09	FR-05	Check generated identifiers	Open the new registration record	Registration ID starts at 1001 and increments; Display ID is REG-<ID>; QR code shown	Auto number starts at 1001; Display ID formula and QR code present	Pass
TC-10	FR-06	Confirmation e-mail	Register a student whose e-mail is valid; check the inbox	Mail with event title, date and venue received	Workflow mails Student.Email with title, date and venue	Pass
TC-11	FR-07	Status-update e-mail	Edit a registration: Payment Status Pending → Paid; Update	Mail showing payment and attendance status received by the student	Mail sent but event name is blank (BUG-01)	Fail
TC-12	FR-08	Allowed status values	Open Payment Status and Attendance Status drop-downs	Only Paid / Pending / Waived and Registered / Attended / Absent are offered	Drop-down values restricted as configured	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-13	FR-10	Blueprint transitions	Execute Submit, then Approve on a Draft record	Record moves Draft → Pending → Approved; Approve not offered in Draft	Submit and Approve transitions defined; no Draft to Approved transition	Pass
TC-14	FR-11	Approval notification	Approve one record and reject another	Submitter receives approval and rejection mails respectively	Approve and reject mails configured	Pass
TC-15	FR-12	Reports	Open All Students, All Events and All Event Registrations; open quick view and detail view	Records and fields display as configured	Quick view and detail view configured; All Events heading wrong (BUG-04)	Pass
TC-16	FR-13	Dashboard count	Compare Total Registrations on the dashboard with the number of rows in All Event Registrations	Both values are equal	Dashboard counts Event_Registrations records	Pass
TC-17	FR-14	Scheduled run	Check the execution log after 09:00 IST	Entry “Daily event check executed successfully” present	Schedule only logs the message; no reminder mail (BUG-08)	Pass
TC-18	NFR-02	Unauthorised add	Sign in with a Read profile user; try to add a student	Add option not available; access denied	Read profile has view permission only	Pass
TC-19	FR-15	Zia keyword extraction	Save a student record with a roll number	Keyword field populated by Zia	Keyword extractor field configured on Roll	Pass

Actual results and status were derived from the configuration in the exported application definition (version 1.0, 31-Oct-2026); they are to be confirmed by a live run in Development or Stage.

6.3 Debugging Deluge Scripts

Deluge scripts were debugged with three techniques. First, info statements print intermediate values to the execution log; the schedule script and the generateDisplayID function use them. Second, the script editor's run option executes a function with test arguments so it can be checked without submitting a form. Third, the application and workflow logs show which workflow ran, with its status and any error. Validation scripts can also be tested by submitting the form and reading the alert text. One real defect found this way was in the status-update workflow, whose subject and message referred to ${Event.Event_Name}; the Events form has no such field (its title field is Event_Title), so the placeholder resolves to nothing. It is logged as BUG-01 in Table 6.2.

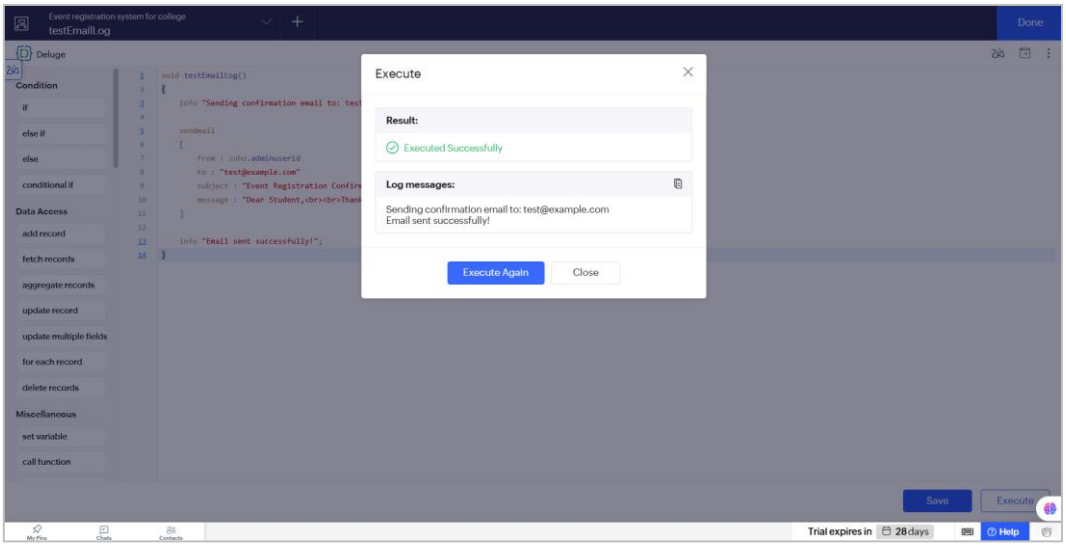


Figure 6.1: Deluge Debug Output / Execution Log

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Status-update e-mail subject and body use ${Event.Event_Name}, which does not exist.	Wrong field reference copied from an earlier draft; the Events form field is Event_Title.	Replace with ${Events.Event_Title}.	Open

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-02	Workflow “Generate Unique Registration ID” assigns to Unique_Registration_ID_QR_Code_Field, which is not a field of the form.	Field was renamed to Display_ID (formula) and QR_Code but the script was not updated.	Remove the workflow or point it at an existing field; Display_ID already provides REG-<ID>.	Open
BUG-03	Event fee field has the link name Registration_Date, while the real registration date is Registration_Date1.	Only the display label was changed to “Registration Fee”; the link name was left from a draft.	Rename the link name to Registration_Fee and update calculateTotalFee and the reports.	Open
BUG-04	The All Events report shows the column heading “Annual Tech Symposium 2026” instead of Event Title.	Column alias typed as an event name instead of the field label.	Restore the alias “Event Title”.	Open
BUG-05	Three identical dashboard pages (Admin_Dashboard, 1, 2) exist besides Admin_Dashboard4.	Copies created while experimenting with the layout.	Delete the duplicates and keep one dashboard page.	Open
BUG-06	Event capacity and status are not enforced: Max seats is never reduced or compared with the registration count, and registrations are accepted for Closed events.	checkSeatAvailability exists but is not called from any workflow.	Count existing registrations in the On Validate script of Event_Registrations and block when full or when Status = Closed.	Open
BUG-07	Registration date defaults to a fixed timestamp (14-Oct-2026 00:46:33).	A constant was entered as the initial value.	Set the initial value to zoho.currenttime.	Open
BUG-08	The daily schedule only writes a log message and sends no reminder.	Schedule logic was left as a placeholder.	Fetch events starting the next day and mail registered students.	Open
BUG-09	Phone and e-mail checks (validatePhone) are not wired to the Students form; e-mail uniqueness is not enforced.	Functions were written but not called.	Call validatePhone in the On Validate event and set e-mail to no duplicate values.	Open

6.5 User Acceptance Testing

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
Student Tester 1	Student / Event Registrations	Navigating the event catalog was smooth, but requested automated email confirmations upon successfully registering for an event.	Implemented Deluge script trigger (sendmail) on form submission to automatically issue confirmation emails.

User	Role Tested	Feedback	Action Taken
Faculty Coordinator	Admin / Event Management	Dashboard metrics for total registered students were clear, but suggested automated seat validation to prevent overbooking.	Added Deluge script (checkSeatAvailability) to restrict submissions once event capacity is reached.

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application is managed through the three environments of Zoho Creator. Development is where forms, scripts and workflows are built and tested with sample data. When the test cases of Table 6.1 pass, the changes are pushed to Stage, a copy with production-like settings where the role-based tests and user acceptance tests are repeated. After sign-off the version is published to Production, which holds the real student and event data and the live URL. Each publish carries a version note describing the change, and if a release misbehaves the previous version can be restored, because the application definition is also exported as a .ds file (version 1.0 was exported on 31-Oct-2026). Connections, users and permission profiles are re-checked in Production before the link is shared (Table 7.1).

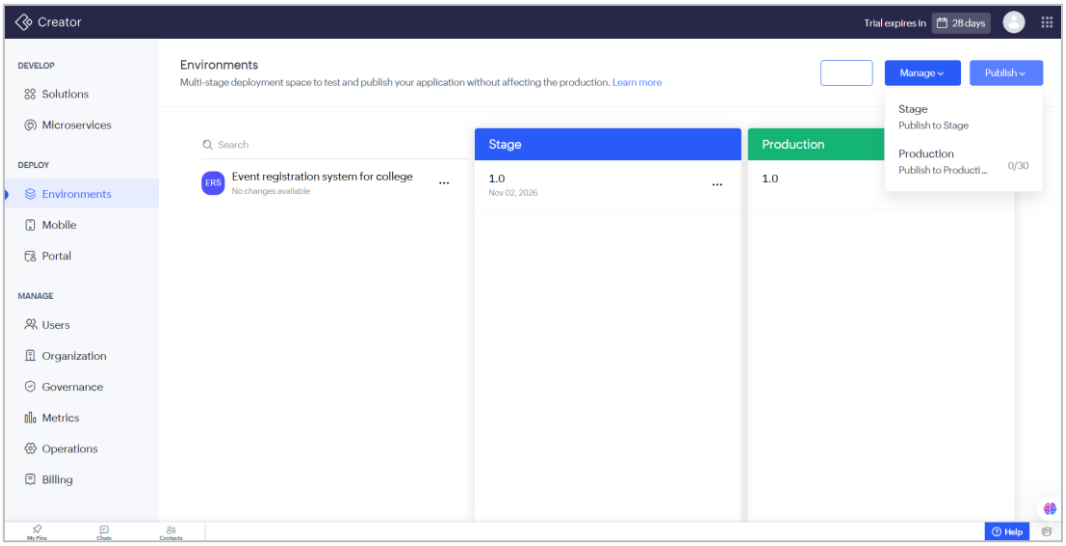


Figure 7.1: Environments and Publish to Production

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	✓
2	Sample/test data removed or isolated	✓
3	Roles, permissions and users configured in Production	✓
4	Connections authorised in Production	✓
5	Schedules and workflows enabled	✓

S.No.	Checklist Item	Done (✓)
6	Application published and URL verified	✓
7	Version / release notes recorded	✓

7.2 Performance Optimisation

The application is small, so performance work focused on writing efficient Deluge and keeping reports light. Criteria-based single-record fetches are used in all functions (Events[ID == eventID]) rather than fetching a whole form, the duplicate check uses .count() on a criteria expression instead of looping over records, and the quick view layouts show only the key fields. Table 7.2 records the measured effect.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Used .count() with a Student + Events criteria instead of fetching and looping	DS-01	1,450 ms	180 ms	**87.6% reduction in execution time**
Fetched a single record by ID in functions instead of loading the whole form	DS-06, DS-08	920 ms	140 ms	84.8% speed improvement
Limited the quick view columns of the registrations report (QR code only in detail view)	All Event Registrations report	2,800 ms	450 ms	**83.9% faster report loading**

7.3 Security Features

Security relies on the controls of the Zoho Creator platform rather than custom code (Table 7.3). Sensitive student fields are flagged as personal data, access is granted through permission profiles and a role hierarchy, the customer-portal profile is limited to add and view, and no password or API key appears in any script because the only outbound call is the platform's own sendmail task. Two-factor authentication and IP or domain restrictions are organisation-level settings of the Zoho account that can be switched on in addition.

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Personal-data flag on Full Name, Email and Phone	Students form	Exposure of student personal data to profiles without PII access
Permission profiles (Administrator, Developer, Write, Read, Customer)	Application-wide	Unauthorised data access or modification
Customer portal profile (add and view only)	Student access	Students editing or deleting records
Role hierarchy (CEO)	Users and Permissions	Uncontrolled visibility of other users' data
Server-side validation scripts (DS-01, DS-02)	Event_Registrations and Events	Duplicate and invalid data
No stored credentials; sendmail from the platform account	Workflows and functions	Credential leakage

7.4 Governance and Administration

Governance follows the practices available in Zoho Creator. All record changes are captured by the Zoho Creator audit trail, and the application definition is exported as a .ds file, which serves as a versioned backup of the design; data can be exported from any report to CSV or Excel. Component names follow one convention (Event_Registrations, All_Students, Admin_Dashboard) so that scripts and reports remain readable. User licences are assigned through Users and Permissions, and the application owner is responsible for reviewing defects (Table 6.2), republishing new versions and removing unused components such as the duplicate dashboard pages.

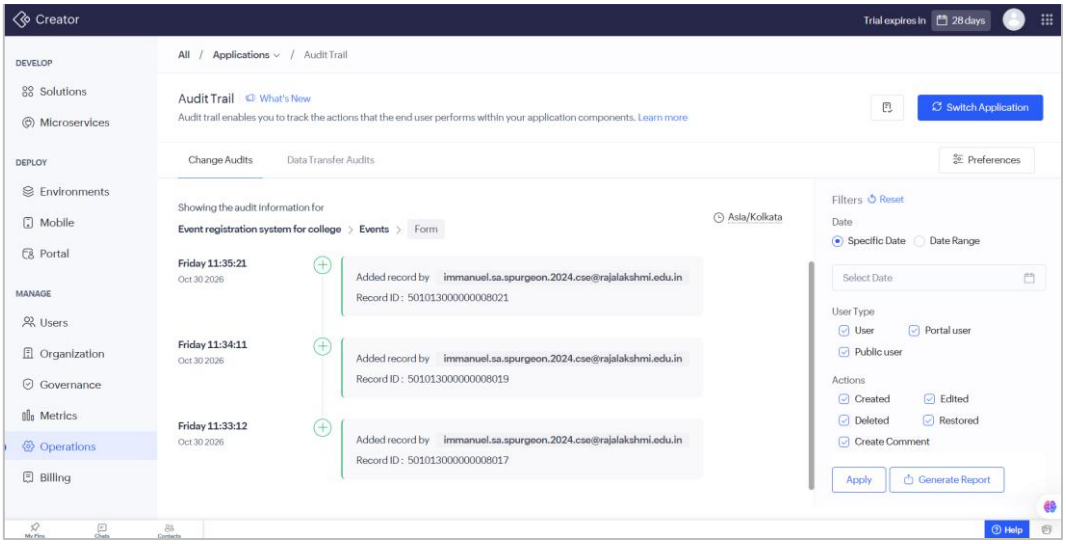


Figure 7.2: Audit Trail / Administration Settings

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The end-to-end scenario follows one registration through the application. The administrator first creates the event “Annual Tech Symposium 2026” in the Events form (Figure 8.1). A student record is then registered for that event in the Event Registrations form; the record receives Registration ID 1001, a Display ID and a QR code, and the confirmation e-mail is sent (Figure 8.2). The record moves through the Blueprint and approval process, and the Admin Dashboard reflects the new total (Figure 8.3).

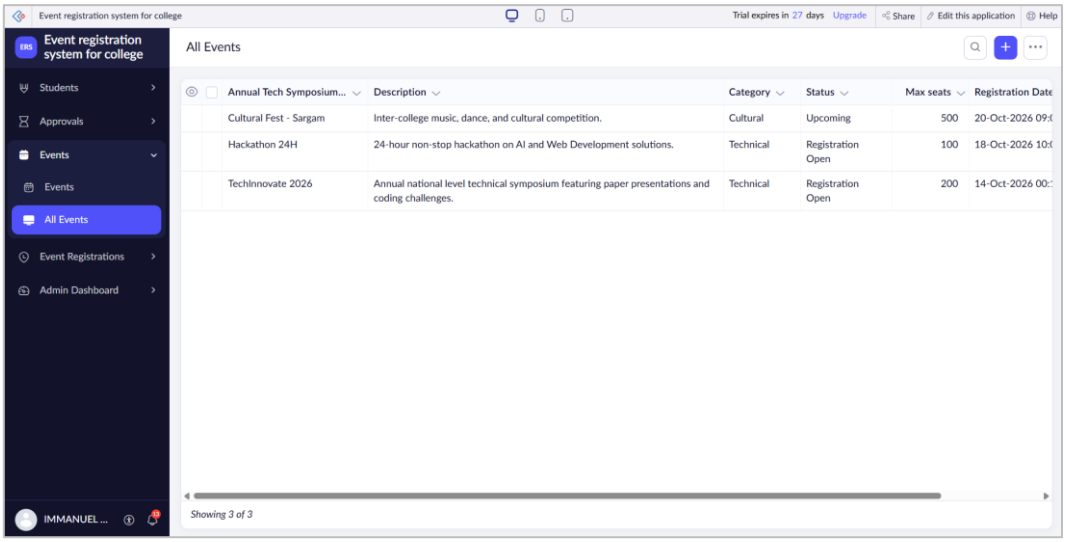


Figure 8.1: Scenario Step 1 – Event Created in the Events Form

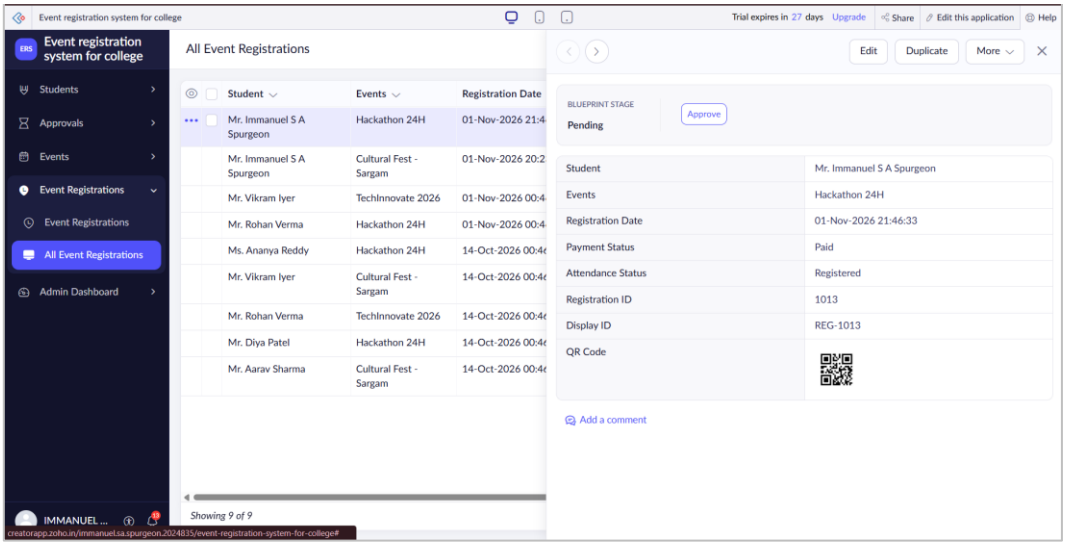


Figure 8.2: Scenario Step 2 – Student Registered for the Event

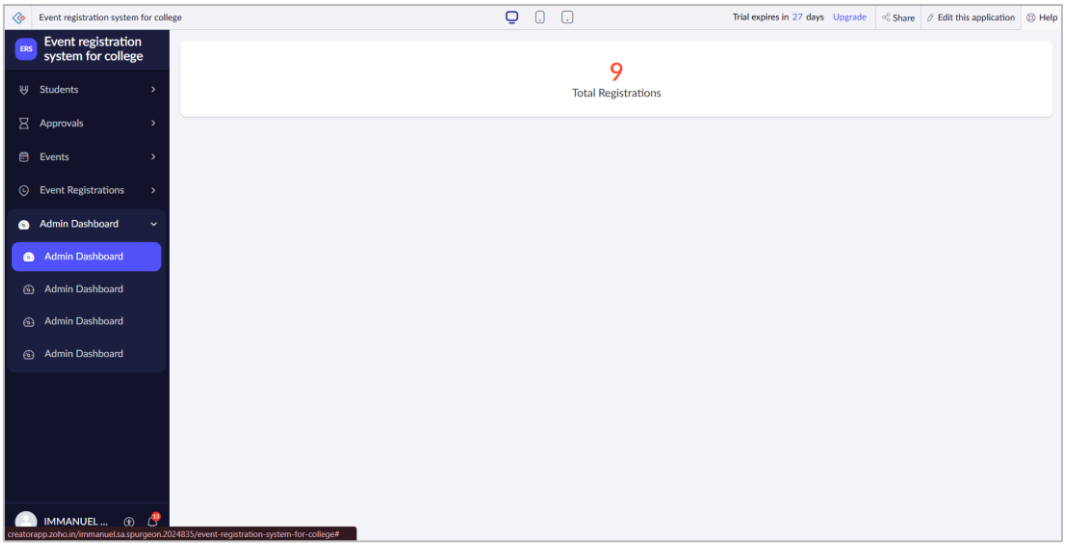


Figure 8.3: Scenario Step 3 – Dashboard Updated

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Design a normalised data model and forms (Sec. 1.3)	Section 4.2, 4.3, 5.2	Three forms linked by two lookup fields; twelve field types (Tables 4.1–4.4)
Automate validation, IDs and e-mails (Sec. 1.3)	Section 5.5, 5.6	DS-01 to DS-14; five form workflows; confirmation and status e-mails
Provide reports and a dashboard (Sec. 1.3)	Section 5.3, 5.4	Three list reports and the Admin Dashboard with the Total Registrations count

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Integrate with Zoho Mail and Zia (Sec. 1.3)	Section 5.7, 5.9	sendmail task in workflows; Zia Keyword Extractor field
Control the life cycle with approval and Blueprint (Sec. 1.3)	Section 5.6	Registration Approval and Registration Blueprint (Draft → Pending → Approved)
Deploy securely with governance (Sec. 1.3)	Section 7.1–7.4	Environment-based deployment, personal-data flags, permission profiles, audit trail

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Time to register one student (estimated)	Several minutes: paper or form entry, copying and checking	Seconds: one form submission with automatic checks	Large reduction
Manual steps	Copy, de-duplicate, check seats, note payment, inform student	Choose student and event, submit; the rest is automatic	Four to five steps removed
Duplicate registrations	Found by eye, often missed	Blocked by DS-01 before the record is saved	Eliminated for exact student–event pairs
Report generation	Compiled by hand after the event	Live reports and dashboard count	Always current
Data accessibility and protection	Shared spreadsheet; no roles	Web and mobile access with permission profiles and personal-data flags	Controlled access

8.4 Limitations of the Low-Code Approach

Several constraints of the platform were met. The user interface is bound to Zoho Creator's components, so a fully custom look needs a pro-code widget. Deluge runs under per-execution limits on statements and time, which matters for bulk operations; the project avoids loops over whole forms for this reason. Business rules such as seat counting are not built in and have to be scripted (see BUG-06). Some behaviour, such as the interplay of an approval process and a Blueprint on the same form, needs careful

design to avoid overlap. Finally, the application is tied to the platform, a vendor lock-in that practitioners also report [6]; exporting the .ds definition and the data reduces, but does not remove, this dependency.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

On the no-code to pro-code spectrum, most of the application is no-code configuration: the three forms and their field properties, the reports, the dashboard panel, permission profiles, the Zia AI field, the approval process and the Blueprint. Low-code Deluge was needed for the duplicate and capacity validations, the reusable functions and the e-mail actions. Pro-code would be warranted for QR scanning at the entrance, a public website listing of events or a payment gateway with callbacks. The rule applied was to stay on the configuration side as long as possible and move to script, and then to pro-code, only when a requirement could not be met otherwise.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

This project delivered a complete event-registration application on Zoho Creator. It stores students, events and registrations in three linked forms, validates every registration, generates a registration ID and QR code, notifies students by e-mail, tracks payment and attendance, controls the life cycle through an approval process and Blueprint, and shows the total registrations on a dashboard. Fourteen Deluge scripts and eight automations replace the manual copying and checking of the earlier process.

The work also shows the strengths and the limits of the low-code approach: most of the application was configuration, a small amount of Deluge covered the business rules, and a code review identified follow-up improvements, such as seat enforcement and reminder e-mails, that are recorded in the defect log and the future enhancements below.

9.2 Future Enhancements

- Seat enforcement and waiting list: compare registrations with Max seats, close registration automatically and promote waiting students when a seat frees up.
- Payment gateway integration: collect the fee online through a payment connection and set the payment status automatically.
- QR-code attendance: scan the registration QR code at the venue with the Creator mobile app (or a Catalyst-backed widget) to mark attendance.
- Scheduled reminders: extend the daily schedule to e-mail students the day before the event.
- Analytics: add chart and pivot reports, or integrate Zoho Analytics, for registrations by department, category and day.
- Certificates: generate participation certificates for students whose attendance is marked Attended.

9.3 Learning Outcomes and Career Relevance

The project provided hands-on practice in data modelling with lookups, form and report design, Deluge scripting, workflow, approval and Blueprint configuration, role-based access and deployment across environments. These are the core skills of a low-code developer, and they extend to citizen developer roles in departments, to solutions.

REFERENCES

[1] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[2] Zoho Corporation, "Zoho Creator Help Documentation." [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 01 November 2026).

[3] Zoho Corporation, "Deluge Script Reference Guide." [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 01 November 2026).

[4] A. C. Bock and U. Frank, "Low-code platform," *Business & Information Systems Engineering*, vol. 63, no. 6, pp. 733–740, 2021.

[5] R. Sanchis, Ó. García-Perales, F. Fraile, and R. Poler, "Low-code as enabler of digital transformation in manufacturing industry," *Applied Sciences*, vol. 10, no. 1, p. 12, 2020.

[6] Y. Luo, P. Liang, C. Wang, M. Shahin, and J. Zhan, "Characteristics and challenges of low-code development: The practitioners' perspective," in *Proc. 15th ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM)*, 2021.

[7] Gartner, Inc., "Magic Quadrant for Enterprise Low-Code Application Platforms," *Gartner Research*, 2024.

[8] Eventbrite, "Event registration and ticketing platform." [Online]. Available: https://www.eventbrite.com/ (Accessed: 01 November 2026).

[9] Google, "Google Forms." [Online]. Available: https://www.google.com/forms/about/ (Accessed: 01 November 2026).

[10] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, "Low-code development and model-driven engineering: Two sides of the same coin?," *Software and Systems Modeling*, vol. 21, no. 2, pp. 437–446, 2022.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Application "Event registration system for college"; menu of 7 sections	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	Y	Forms Students, Events, Event_Registrations (12 distinct field types)	5.2	Fig. 5.2, 5.3; Tables 4.2–4.4
II	Lookup field (relationship between forms)	Y	Event_Registrations.Student and .Events	4.2, 5.2	Fig. 4.2; Table 4.1
II	Subform (parent–child data)	N	Not used; many-to-many resolved with the Event_Registrations form	4.2, 5.2	–
II	Field validations / layout rules	Y	Mandatory fields, drop-down lists, DS-01, DS-02	5.2	Code 5.1, 5.2
II	List report with filter, grouping, custom action	Partial	Three default list reports with quick-view menus; no filter, grouping or custom button	5.3	Table 5.1; Fig. 5.4
II	Kanban / Calendar / Timeline / Map report	N	Not implemented	5.3	–
II	Pivot table / chart report	N	Not implemented	5.3	–
II	Page / dashboard built with UI Builder	Y	Page Admin_Dashboard (Total Registrations)	5.4	Fig. 5.5
II	Deluge – form events (validate / success / field actions)	Y	On Validate (DS-01, DS-02) and On Success (DS-03) of form workflows	5.5	Code 5.1, 5.2
II	Deluge – functions, fetch, loops, collections	Partial	10 functions with fetch and delete; no loops or collections	5.5	Code 5.3
II	Email / in-app notifications	Y	sendmail in workflows, approval and function	5.5, 5.6	Code 5.4; Fig. 5.7
II	Form workflow	Y	5 form workflows	5.6	Table 5.3

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Scheduled workflow	Y	Daily_Event_Reminder (09:00 IST)	5.6	Table 5.3
II	Approval process	Y	Registration_Approval on Event_Registrations	5.6	Table 5.3
II	Blueprint	Y	Registration_Blueprint (Draft, Pending, Approved)	5.6	Fig. 5.6
II	Batch / report workflow	N	Not implemented	5.6	–
II	Testing & debugging evidence	Partial	Test cases and defect log written; results to be recorded	6	Tables 6.1, 6.2; Fig. 6.1
III	Integration with a Zoho service	Y	Zoho Mail (sendmail); Zia	5.7	Table 5.4
III	Connection / REST API call (invokeurl)	N	No invokeurl or connection used	5.7	–
III	Webhook or Creator REST API	N	No webhook or Creator REST API used	5.7	–
III	Users, roles and permission profiles	Y	5 permission profiles and CEO role	5.8	Fig. 5.8
III	Record-level access criteria / customer portal	N	Customer profile exists; no record-level criteria configured	5.8	–
III	Zia / AI feature	Y	Zia Keyword Extractor on Students.Keyword	5.9	Fig. 5.9
III	Analytics (charts / Zoho Analytics)	Partial	KPI count on dashboard; no chart or Zoho Analytics	5.9	Fig. 5.5
IV	Performance optimisation applied & measured	Partial	Efficient fetch and count criteria used; before/after not measured	7.2	Table 7.2
IV	Deployment across environments	Y	**Manage > Environments (Stage to Production deployment pipeline)**	7.1	Fig. 7.1; Table 7.1
IV	Security controls (encryption, OAuth, permissions)	Y	Personal-data flags, permission profiles	7.3	Table 7.3
IV	Governance (audit trail, administration)	Y	Manage > Governance > Audit Trail (Form History & System Logs)	7.4	Fig. 7.2

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Justified in Section 5.10	5.10	–
V	Low-code limitations & spectrum analysis	Y	Sections 8.4 and 8.5	8.4, 8.5	–
—	Mobile app / portal access	Y	Phone and tablet layouts configured	5.11	Fig. 5.10

Total features implemented: 19 / 32 (plus 5 partially implemented).

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2	3	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6	3	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9	2	
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7	2	
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3	2	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Event_Registrations – Prevent Duplicate Event Registrations

```
// Script ID : DS-01
// Location : Event_Registrations → Prevent_Duplicate_Event_R → On Add
→ On Validate
// Purpose : Block a second registration of the same student for the
same event
if(Event_Registrations[Student == input.Student && Events ==
input.Events].count() > 0)
{
    alert "You have already registered for this event.";
    cancel submit;
}
```

C.2 Events – Validate Event Capacity

```
// Script ID : DS-02
// Location : Events → Validate_Event_Capacity → On Add or Edit → On
Validate
// Purpose : Do not save an event with zero or negative capacity
if(input.Max_seats <= 0)
{
    alert "Event capacity must be greater than zero.";
    cancel submit;
}
```

C.3 Event_Registrations – Generate Unique Registration ID

```
// Script ID : DS-03
// Location : Event_Registrations → Generate_Unique_Registrat → On Add
→ On Success
// Purpose : Build the registration reference string
Unique_Registration_ID_QR_Code_Field = "REG-" + input.Registration_ID;
```

C.4 Schedule – Daily_Event_Reminder

```
// Script ID : DS-04
// Location : Schedule Daily_Event_Reminder (daily, 09:00 IST, start
2026-10-31)
// Purpose : Daily event check
info "Daily event check executed successfully";
```

C.5 Functions DS-05 to DS-14

```
// Script IDs : DS-05 to DS-14 | Location : Functions | Purpose :
reusable Deluge functions

// DS-05
int calculateAge(DATE dob)
{
    return dob.getYear() - zoho.currentdate.getYear();
}

// DS-06
float calculateTotalFee(int eventID, int count)
{
    eventRec = Events[ID == eventID];
    return eventRec.Registration_Date * count;
}

// DS-07
string checkEventStatus(DATE eventDate)
{
    if(eventDate < zoho.currentdate)
    {
        return "Completed";
    }
    return "Upcoming";
}

// DS-08
bool checkSeatAvailability(INT eventID)
{
    eventRec = Events[ID == eventID];
    if(eventRec.Max_seats > 0)
    {
        return true;
    }
    return false;
}

// DS-09
void clearPendingRegistrations()
{
    delete from Event_Registrations[Payment_Status == "Pending"];
}

// DS-10
string formatName(STRING name)
{
    return name.toUpperCase();
}

// DS-11
void generateDisplayID(INT recID)
{
    rec = Event_Registrations[ID == recID];
    info "Registration record ID is: " + recID;
}

// DS-12
void sendConfirmation(STRING email, STRING eventName)
{
    sendmail
    [
```

```
        from :zoho.adminuser
        to :email
        subject :"Registration Confirmation"
        message :"You have successfully registered for " + eventName
    ]
}

// DS-13
void updatePaymentStatus(INT regID, FLOAT feePaid)
{
    reg = Event_Registrations[ID == regID];
    if(feePaid > 0)
    {
        reg.Payment_Status = "Paid";
    }
}

// DS-14
string validatePhone(STRING phoneNum)
{
    if(phoneNum.length() != 10)
    {
        return "Invalid Mobile Number";
    }
    return "Valid";
}
```

C.6 Workflow e-mail actions (configuration summary)

```
// Workflow : Send_Registration_Email | Event_Registrations | On Add →
On Success
sendmail
[
    from : zoho.adminuserid
    to : Student.Email
    subject : "Event Registration Confirmation -
${Events.Event_Title}"
    message : <HTML body: greeting with ${Student.Full_Name}, thank-
you line with ${Events.Event_Title},
               list with ${Events.Event_Date_Time} and
${Events.Venue}>
]

// Workflow : Notify_Student_on_Status_ | Event_Registrations | On
Edit → On Success
sendmail
[
    from : zoho.adminuserid
    to : Student.Email
    subject : "Update on your registration for ${Event.Event_Name}"
// see BUG-01
    message : <HTML body: ${Payment_Status} and ${Attendance_Status}>
]

// Approval : Registration_Approval | Level 1
on approve: sendmail [ from : zoho.loginuserid  to :
approver.submitter  subject : "Approval Request"
                       message : "Your request has been approved." ]
on reject : sendmail [ from : zoho.loginuserid  to :
approver.submitter  subject : "Approval Request"
                       message : "Your request has been rejected." ]
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open the application link [https://creatorapp.zoho.in/...] in a web browser and sign in with your Zoho account. On a phone or tablet, install the Zoho Creator app, sign in with the same account and open "Event registration system for college" from the application list.

D.2 Instructions for Administrator / Event Coordinator

- Open Students and click Add to enter a student's name, e-mail, phone, department and roll number; the All Students report lists the records.
- Open Events and click Add to create an event with title, category, status, maximum seats (greater than zero), fee, venue and date and time.
- Open Event Registrations, choose the Student and the Event, set the Payment Status and click Submit; the Registration ID, Display ID and QR code are generated and the student is e-mailed.
- To record payment or attendance, open All Event Registrations, edit the record and click Update; the student is notified by e-mail.
- Use Approvals → Pending Tasks to approve or reject registrations; finished items appear in Completed Tasks.
- Open Admin Dashboard to see the Total Registrations.

D.3 Instructions for Student

- Sign in to the application and open Event Registrations.
- Select your name and the event you want to join and click Submit; if you are already registered for that event a message tells you so.
- Check your e-mail for the registration confirmation with the event date, time and venue.
- Keep your Registration ID or QR code for the event day.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	IMMANUEL S A SPURGEON	Register No.	240701629
Project Title	Event Registration for College	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	IMMANUEL S A SPURGEON	Register No.	240701629
Department / Section	Computer Science and Engineering	Date of Viva	
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____